

SE4040 – Enterprise Application Development

Assignment 1

Smart Solar Microgrid Trading System



IT Number	Student Name
IT23452916	M.L.BOOSO
IT23151260	G.L.S.CHANLAKA
IT23156760	L.T.JAYAWARDHANA
IT23192300	H.N.MADUBASHINI

Git Repository Link :- <https://github.com/SakithLiyanage/Solvance-Smart-SolarMicrogrid-Trading-System>

Submission date :- October 05, 2026

Table of Contents

1. Introduction.....	3
1.1. Project Overview	3
1.2. Objectives	3
1.3. System Scope.....	4
2. High-Level System Architecture.....	5
3. Use Case Diagram.....	6
4. Level 1 Data Flow Diagram.....	7
5. Database Design.....	8
6. User Interface.....	9
6.1. Web Application Screens	9
6.2. Android Application Screens	19
7. Source Code / Implementation.....	24
7.1. M.L. Booso (IT23452916) - User Identity & Prosumer Lifecycle	24
7.2. G.L.S. Chanlaka (IT23151260) - Solar Nodes & Station Geolocation	31
7.3. L.T. Jayawardhana (IT23156760) - Energy Reservations & Rule Engine	41
7.4. H.N. Madubashini (IT23192300) - Operator Terminal, QR & Telemetry	60
8. Individual Contribution.....	68
9. Challenges.....	70
9.1. Challenges Faced	70
9.2. Solutions	70
10. Conclusion	71
11. References.....	72
12. Appendix.....	73

1. Introduction

1.1. Project Overview

The Smart Solar Microgrid Trading System is an enterprise client-server application designed to facilitate the management and trading of solar energy through microgrid nodes. The system consists of a web application for Backoffice users and Grid Operators, a native Android mobile application for Solar Prosumers and Grid Operators, and a centralized C# Web API that manages the system's business logic and server-side data.

The web application provides administrative and operational functionality, including user management, prosumer management, microgrid node management, and energy reservation management. Solar Prosumers use the mobile application to register and manage their accounts, reserve energy drop-off or charging slots, modify or cancel reservations, view booking information and history, and locate nearby microgrid nodes using Google Maps. Grid Operators can access both the web and mobile applications to monitor bookings, manage operational activities, scan prosumer transaction QR codes, verify transaction information, and finalize energy transfers.

The system follows a client-server architecture in which the web and Android applications communicate with the centralized Web API through RESTful API calls. The C# Web API follows the FAT Service pattern, where the business logic is centralized within the service rather than being implemented independently in the client applications. The Web API is hosted on a Windows IIS server and uses a NoSQL database such as MongoDB for server-side data storage. The Android application additionally uses SQLite for the required local persistence.

1.2. Objectives

The main objective of the Smart Solar Microgrid Trading System is to develop an end-to-end enterprise application that enables solar energy reservations and operational management through integrated web and mobile clients.

The specific objectives of the system are to:

- Provide role-based access for Backoffice users and Grid Operators through the web application.
- Allow Backoffice users to create, update, deactivate, and manage prosumer accounts.
- Allow the management of solar microgrid nodes, including their GPS locations, capacity specifications, battery storage availability, and operational schedules.
- Provide energy slot reservation functionality for creating, updating, and cancelling power trading reservations.
- Enforce the specified reservation business rules, including the seven-day scheduling restriction and the twelve-hour notice requirement for reservation updates and cancellations.
- Enable Solar Prosumers to register and manage their accounts through the native Android application.

- Allow Prosumers to create, modify, cancel and review energy reservations through the mobile application.
- Provide dashboards, booking history, pending bookings and booking search functionality.
- Display nearby microgrid nodes using the Google Maps API.
- Generate transaction QR codes for approved reservations and allow Grid Operators to scan and verify these transactions.
- Allow Grid Operators to finalize energy transfer operations after successful server-side verification.
- Centralize business logic and data processing within the C# Web API while allowing the web and Android applications to function primarily as client interfaces.
- Use MongoDB as the server-side NoSQL database and SQLite for required local persistence within the Android application.

1.3.System Scope

The scope of the system covers the development and integration of three main components: the Web Application, Native Android Application, and Centralized Web Service.

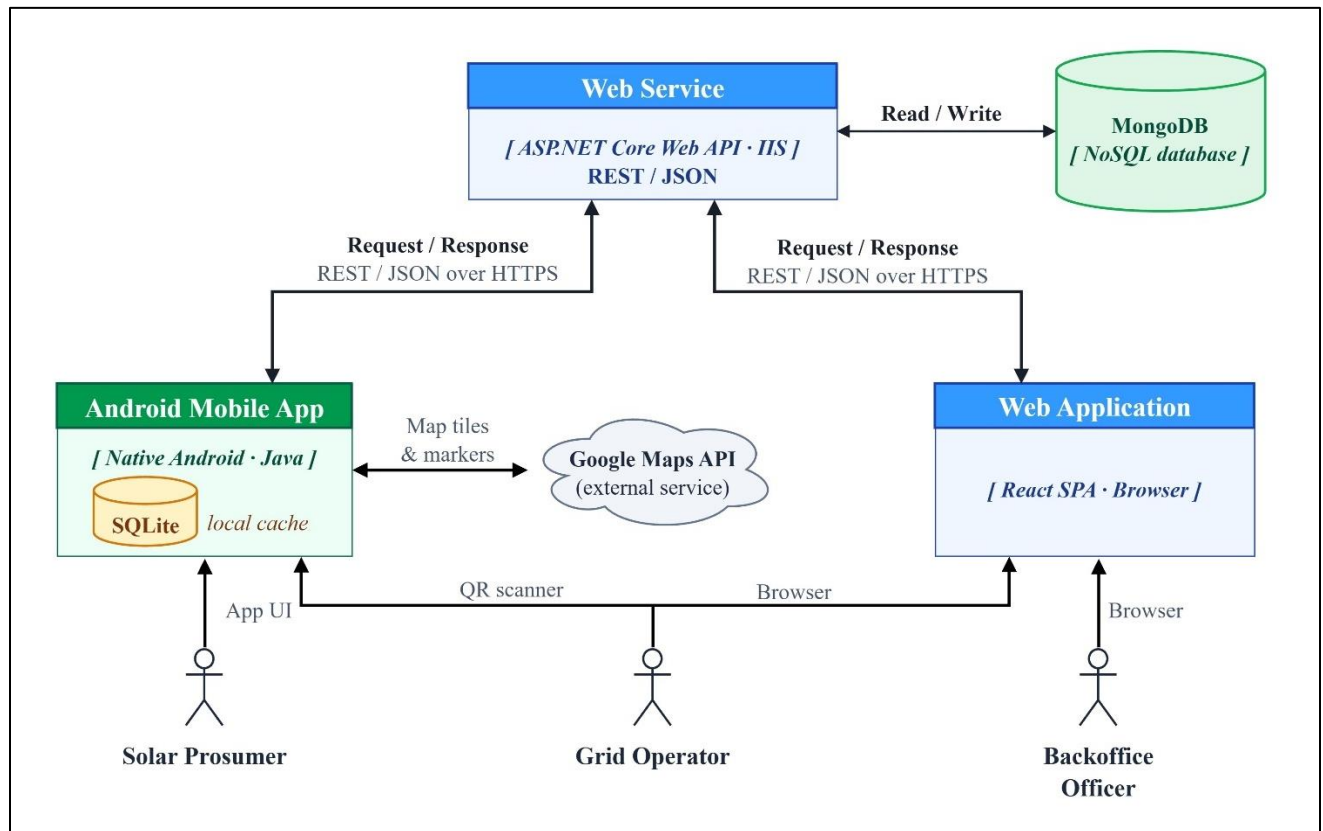
The Web Application is intended for Backoffice users and Grid Operators. Backoffice users can manage web users, prosumers, microgrid nodes, and energy reservations. Grid Operators can use the web application to monitor and manage operational information related to power trading bookings.

The Native Android Application supports both Solar Prosumers and Grid Operators. Prosumers can register and manage their accounts, create and manage reservations, view booking information and history, search bookings, and locate nearby microgrid nodes. Grid Operators can use the mobile application to scan transaction QR codes, verify booking information against the server, and finalize energy transfer operations. The Android application uses SQLite for local persistence and integrates the Google Maps API for displaying microgrid node locations.

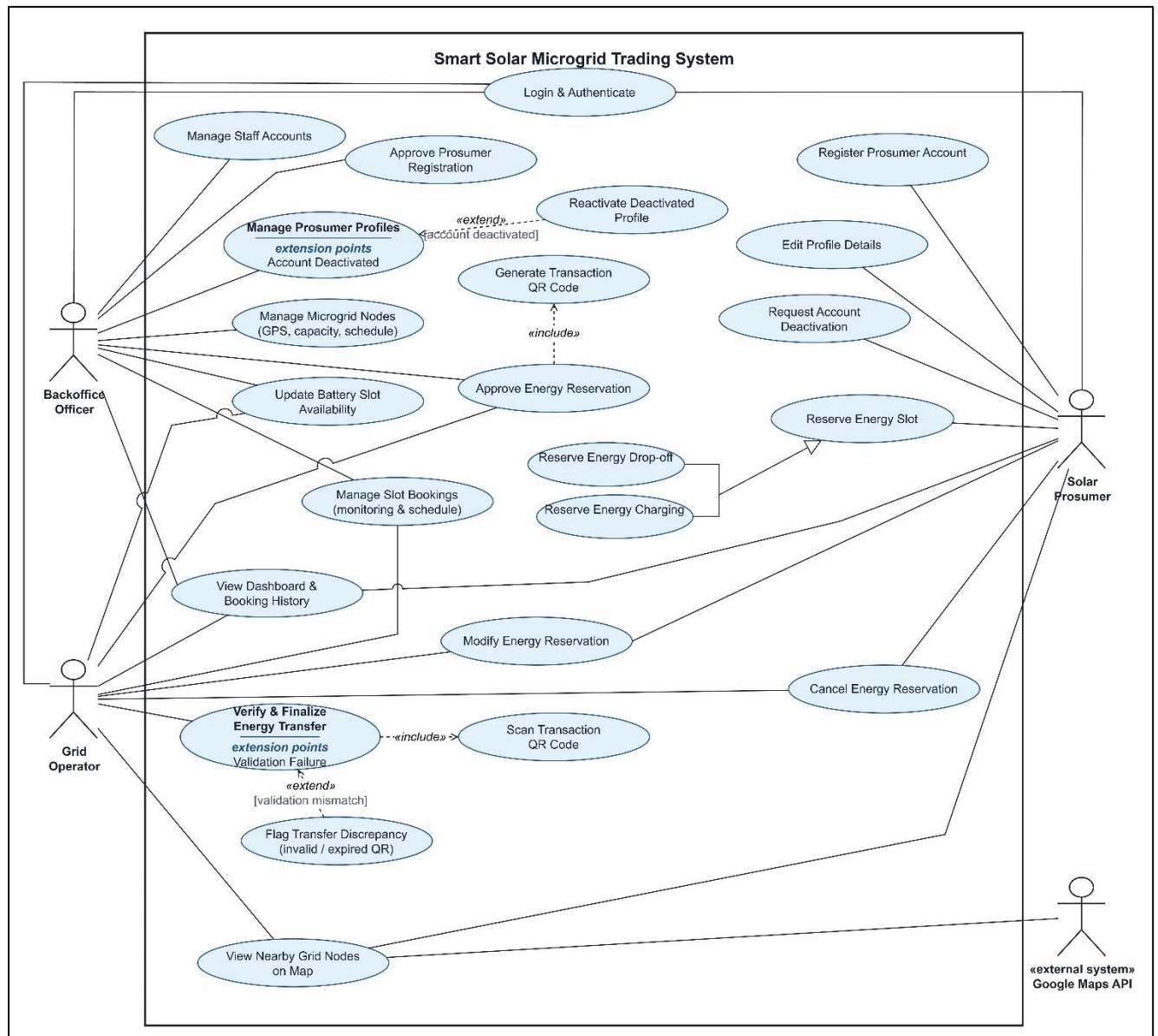
The Centralized Web API acts as the core of the system. It provides RESTful services to both client applications and is responsible for implementing the system's business rules and communicating with the server-side NoSQL database. The service is developed using C# Web API and deployed on Windows IIS. The clients do not directly access the server-side database; instead, they communicate with the Web API to retrieve and process system data.

The system therefore covers the complete flow from user and prosumer management through microgrid node management and energy reservation to QR-based transaction verification and energy transfer completion. The implementation is limited to the functionality specified in the assignment, including the required web, mobile, API, database, local persistence, mapping, and QR-code capabilities.

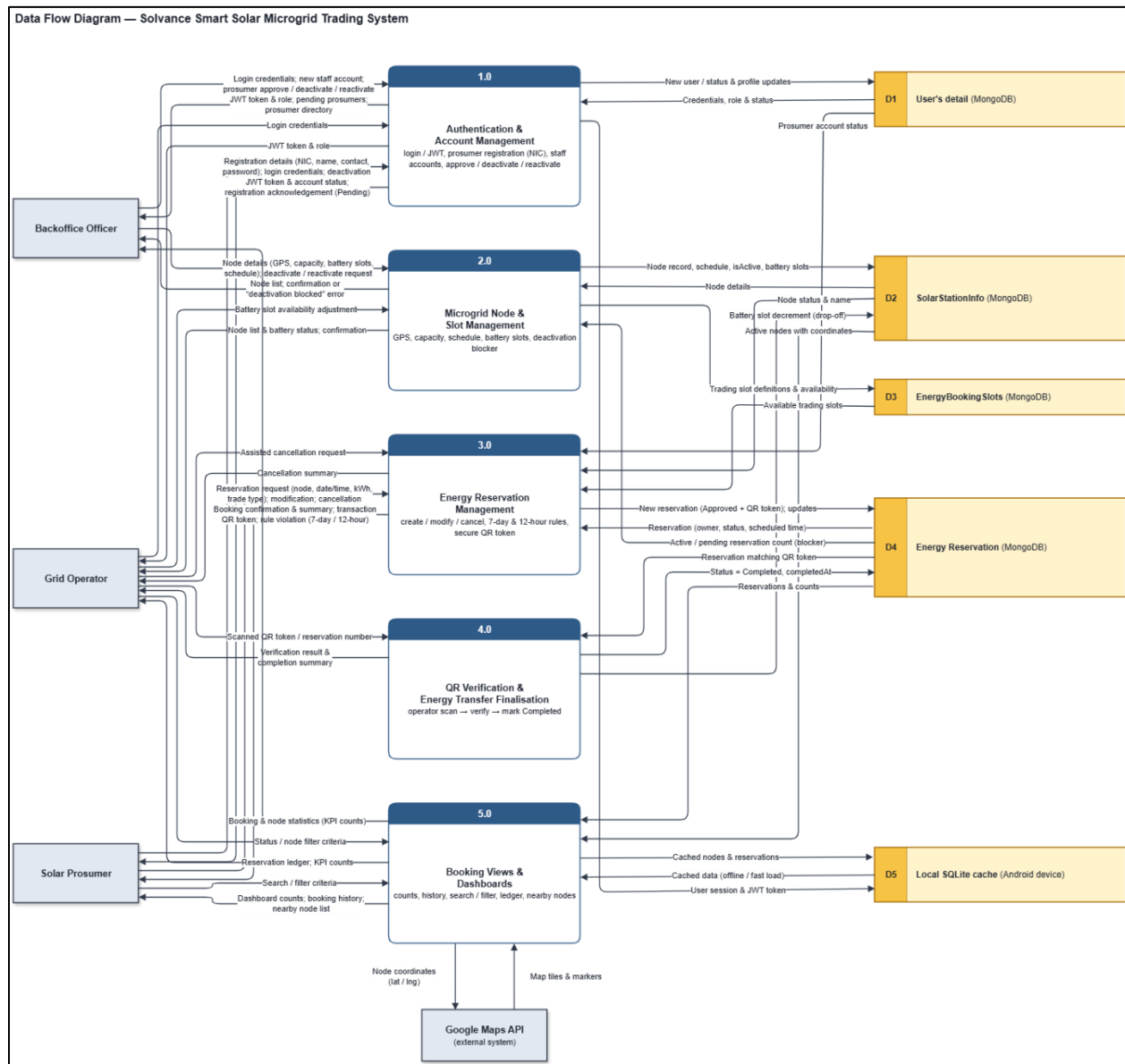
2. High-Level System Architecture



3. Use Case Diagram

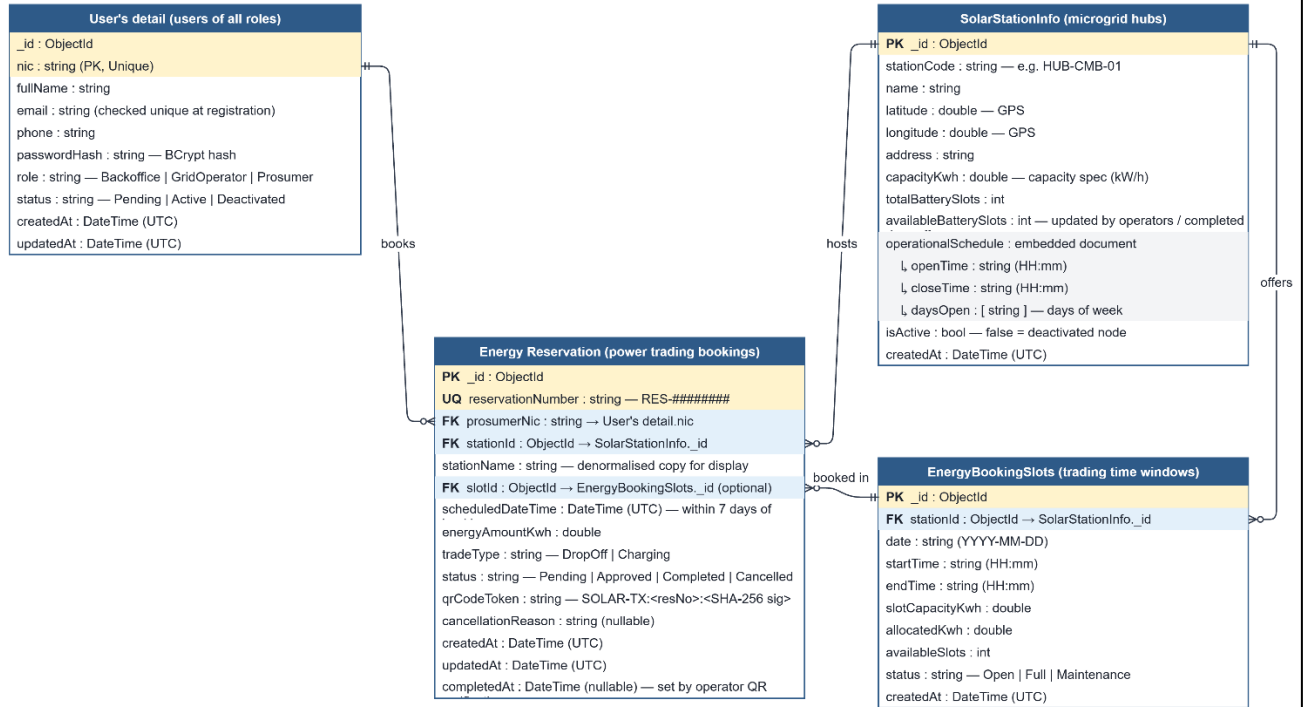


4. Level 1 Data Flow Diagram



5. Database Design

Database Design - MongoDB collections (SolarMicrogridDb) and Android SQLite cache



Android local persistence — SQLite *SolarMicrogridLocal.db* (session & cache only; MongoDB via the Web API is the single source of truth)

user_session	cached_stations	cached_reservations
PK nic TEXT full_name TEXT email TEXT phone TEXT role TEXT status TEXT jwt_token TEXT — bearer token for API calls	PK id TEXT — SolarStationInfo._id station_code TEXT name TEXT latitude REAL longitude REAL address TEXT capacity_kwh REAL total_slots INTEGER available_slots INTEGER	PK id TEXT — Energy Reservation._id reservation_number TEXT prosumer_nic TEXT station_name TEXT scheduled_datetime TEXT (ISO 8601) energy_kwh REAL trade_type TEXT status TEXT qr_token TEXT

6. User Interface

6.1.Web Application Screens

[← Back to home](#)[Dark Mode](#)



Staff sign in

For Backoffice and Grid Operator accounts. Prosumers use the Android app.

Work email, Staff ID, or NIC

Password

[Sign In →](#)

**SOLVANCE** Backoffice

OverviewProsumersSolar HubsStaff & OperatorsReservations

SA System Administrator200331713189Logout

Backoffice Overview

Solar hubs, prosumer accounts and energy reservations at a glance.

[Refresh](#)[Staff & Operators →](#)[Manage Prosumers →](#)

Network Summary

Across active solar hubs

Station Capacity

2500 kWh

Combined capacity of active hubs

Battery Slots in Use

1 / 54

2% occupied

Open Reservations

4

Pending or approved

Completed Energy

15.0 kWh

From completed reservations

Active Solar Hubs

2

Manage hubs and battery slots

Pending Prosumers

0

Waiting for Backoffice approval

Active Prosumers

4

Approved and able to reserve energy

Upcoming Reservations

0

Approved + 4 pending approval

Pending Prosumer Approvals

New registrations start as Pending and can't reserve energy until a Backoffice officer approves them.

[View all →](#)



No pending registrations

Every prosumer account has been reviewed.

SOLVANCE • Smart Solar Microgrid Trading

Overview

Prosumers

Solar Hubs

Staff & Operators

Reservations

System Administrator

280331713189

Logout

Staff & Operator Administration

Create and manage web console accounts. Backoffice Admins have full administration access; Grid Operators use the operational tools.

New Staff

3

Grid Operators

2

Backoffice Admins

1

Active Accounts

3/3

Search by ID, name, email, or phone...

Role: All Roles Operators Backoffice

Status: All Active Deactivated

Staff Accounts

3 accounts

Staff Member	Staff ID	Contact	Assigned Hub / Office	Status	Actions
System Administrator	280331713189	admin@solarmicrogrid.lk +94771234567	National Operations Comman...	Active	Deactivate
Malantha	OPERATOR101	malantha@way.lk 0789890965	-	Active	Deactivate
Priyantha Malagamuwa	OPERATOR102	priyantha@opertaor.lk 0789890963	MG-COL-01 – Galle Fort Coas...	Active	Deactivate

Showing 1-3 of 3 staff

Rows per page 10

New Grid Operator Account

Provision a new Grid Operator account for station operations, battery slot allocation, and QR verification.

Staff ID *

e.g., OP-101, ENP-2024, or ADMIN101

Unique staff identifier or employee code

Full Name *

e.g. Priyantha Jayasuriya

Email *

priyantha@solarmicrogrid.lk

Phone *

+9477 987 6543

Assigned Hub *

Select a hub...

Initial Password *

At least 8 characters, letters and numbers

Minimum 8 characters with letters and numbers.

Cancel

Create Account

Overview

Prosumers

Solar Hubs

Staff & Operators

Reservations

System Administrator

280331713189

Logout

Staff & Operator Administration

Create and manage web console accounts. Backoffice Admins have full administration access; Grid Operators use the operational tools.

Refresh

New Operator Account

Total Staff

3

All web console accounts

Grid Operators

2

Operational tools only

Backoffice Admins

1

Full administration access

Active Accounts

3/3

Can currently sign in

Search by ID, name, email, or phone...

Role: All Roles Operators Backoffice

Status: All Active Deactivated

Staff Accounts

3 accounts

Staff Member	Staff ID	Contact	Assigned Hub / Office	Status	Actions
System Administrator	280331713189	admin@solarmicrogrid.lk +94771234567	National Operations Comman...	Active	Deactivate
Malantha	OPERATOR101	malantha@way.lk 0789890965	-	Active	Deactivate
Priyantha Malagamuwa	OPERATOR102	priyantha@opertaor.lk 0789890963	MG-COL-01 – Galle Fort Coas...	Active	Deactivate

Showing 1-3 of 3 staff

Rows per page 10

SOLVANCE

Smart Solar Microgrid Trading

SOLVANCE | Dashboard | Reservations | Staff & Operator Accounts | System Administrator

Staff & Operator Accounts

Create and manage Grid Operator accounts for station operations, battery slot allocation, and QR verification. Operators use the operational tools.

3 Grid Operator Accounts

Staff Accounts | 3 accounts

Staff Member

New Grid Operator Account

Provision a new Grid Operator account for station operations, battery slot allocation, and QR verification.

StaffID *
e.g., OP-101, ENP-2024, or ADMIN101
Unique staff identifier or employee code.

FullName *
e.g. Priyantha Jayasuriya

Email * priyantha@solarmicrogrid.lk **Phone *** +9477 987 6543

Assigned Hub *
Select a hub...

Initial Password *
At least 8 characters, letters and numbers.
Minimum 8 characters with letters and numbers.

Cancel **Create Account**

SOLVANCE | Dashboard | Reservations | Staff & Operator Accounts | System Administrator

Reservation Queue

Manage reservation queue and track reservation status. All reservation details are visible here.

3/3 Reservations

QR Pass Issued



RES-95218332
Galle Fort Coastal Solar Hub
NIC: 280456556578 • Energy: 15 kWh • Type: Drop-Off
Scheduled: Wed, Oct 07, 04:01 AM

QR Token
SOLAR-TX-RES-95218332-CS198878E967

Copy Token **Scan with Webcam** Done

Backoffice

Overview
Prosumers
Solar Hubs
Staff & Operators
Reservations

SA System Administrator
200532723189
Logout

Energy Reservations

Reservations must be scheduled within 7 days. Changes, and cancellations by prosumers, need at least 12 hours' notice.

Refresh
New Reservation

Total Reservations
9

Pending
4

Approved
0

Completed
1

Cancelled
4

Completed Energy
15.0 kWh

All Statuses
All Stations
Any Date

Reservation #	Prosumer NIC	Station	Scheduled Time & 12-Hour Rule	Energy & Type	Status	Actions
RES-95218332	200456556578	Galle Fort Coastal Solar Hub	Wed, Oct 07, 04:01 AM ✓ Editable - 29.7h left	15 kWh Drop Off	Pending	
RES-48277478	200456556578	Galle Fort Coastal Solar Hub	Mon, Oct 12, 09:19 PM ✓ Editable - 167.0h left	15 kWh Drop Off	Pending	
RES-11637664	200456556578	Colombo Harbor Microgrid Substation	Mon, Oct 12, 09:26 PM ✓ Editable - 167.3h left	10 kWh Charging	Pending	
RES-37548067	941234567V	Galle Fort Coastal Solar Hub	Mon, Oct 05, 12:00 PM ⚠ Time passed	10 kWh Drop Off	Pending	
RES-28000100 	200456556578	Galle Fort Coastal Solar Hub	Mon, Oct 12, 08:54 PM	15 kWh Drop Off	Completed	
RES-66839741	941234567V	Galle Fort Coastal Solar Hub	Sat, Oct 10, 11:45 PM	20 kWh Drop Off	Cancelled	
RES-87447403	941234567V	Galle Fort Coastal Solar Hub	Sat, Oct 10, 11:45 PM	20 kWh Drop Off	Cancelled	
RES-59754269	941234567V	Galle Fort Coastal Solar Hub	Sat, Oct 10, 11:45 PM	20 kWh Drop Off	Cancelled	
RES-43772778	941234567V	Galle Fort Coastal Solar Hub	Tue, Oct 06, 11:30 AM	12 kWh Drop Off	Cancelled	

Showing 1-9 of 9 reservations
Rows per page: 10

SOLVANCE

Smart Solar Microgrid Trading

Grid Operator

PM Priyantha Malagamuwa
OPERATOR102
Logout

Grid Operator Dashboard

Logged in as Priyantha Malagamuwa (OPERATOR102) • Operating hub: Galle Fort Coastal Solar Hub

Refresh

Reservation Queue
4 Reservations
Pending & approved, all stations

Verified Trades
1 Completed
Verified by QR scan

Battery Capacity
53 Slots Free
Across all active stations

QR Scanner
Reservation Queue (4)
Battery Slots (29 free)
All Panels

Verify QR Pass

Scan the prosumer's QR pass with the webcam, or enter the token manually

Webcam
Manual Entry

Camera Off

Start the webcam to scan the QR pass on the prosumer's phone.

Start Webcam

QR Token

e.g., SOLAR-TX:RES-78442525:87AF828EED4D or RES-78442525

Verify & Complete

SOLVANCE

Smart Solar Microgrid Trading


SOLVANCE
Grid Operator


Priyantha Malagamuwa
OPERATOR102
Logout

Grid Operator Dashboard

Logged in as **Priyantha Malagamuwa (OPERATOR102)** • Operating hub: **Galle Fort Coastal Solar Hub**

Reservation Queue
4 Reservations
Pending & approved, all stations

Verified Trades
1 Completed
Verified by QR scan

Battery Capacity
53 Slots Free
Across all active stations

QR Scanner
Reservation Queue (4)
Battery Slots (29 free)
All Panels

Reservation Queue

Upcoming reservations first (soonest at the top), then overdue ones, then history.

All (9)
Pending (4)
Approved (0)
Completed (1)
Cancelled (4)

Reservation #	Prosumer NIC	Station & Trade Type	Scheduled Window	Status	Actions
RES-95218332	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Wed, Oct 07, 04:01 AM Editable: 29.6h left	Pending	Approve Edit Cancel
RES-48277478	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 09:19 PM Editable: 166.9h left	Pending	Approve Edit Cancel
RES-11637664	288456556578	Colombo Harbor Microgrid Substation Other Hub 10 kWh • Charging	Mon, Oct 12, 09:26 PM Editable: 167.1h left	Pending	Approve Edit Cancel
RES-37548867	941234567V	Galle Fort Coastal Solar Hub 10 kWh • Drop Off	Mon, Oct 05, 12:00 PM Time passed	Pending	Approve Edit Cancel
RES-28080180	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 08:54 PM	Completed	—
RES-66839741	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-87447483	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-59754269	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-43772778	941234567V	Galle Fort Coastal Solar Hub 12 kWh • Drop Off	Tue, Oct 06, 11:30 AM	Cancelled	Reason: Suhe cleanup

Showing 1-9 of 9 reservations
Rows per page: 10

SOLVANCE • Smart Solar Microgrid Trading


SOLVANCE
Grid Operator

PM
Priyantha Malagamuwa
OPERATOR102
Logout

Grid Operator Dashboard

Logged in as Priyantha Malagamuwa (OPERATOR102) • Operating hub: Galle Fort Coastal Solar Hub

Reservation Queue
4 Reservations
Pending & approved, all stations

Verified Trades
1 Completed
Verified by QR scan

Battery Capacity
53 Slots Free
Across all active stations

QR Scanner
Reservation Queue (4)
Battery Slots (29 free)
All Panels

Reservation Queue

Upcoming reservations first (soonest at the top), then overdue ones, then history.

All (9)
Pending (4)
Approved (0)
Completed (1)
Cancelled (4)

Reservation #	Prosumer NIC	Station & Trade Type	Scheduled Window	Status	Actions
RES-95218332	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Wed, Oct 07, 04:01 AM Editable: 29.6h left	Pending	Approve Edit Cancel
RES-48277478	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 09:19 PM Editable: 166.9h left	Pending	Approve Edit Cancel
RES-11637664	288456556578	Colombo Harbor Microgrid Substation Other Hub 10 kWh • Charging	Mon, Oct 12, 09:26 PM Editable: 167.1h left	Pending	Approve Edit Cancel
RES-37548867	941234567V	Galle Fort Coastal Solar Hub 10 kWh • Drop Off	Mon, Oct 05, 12:00 PM Time passed	Pending	Approve Edit Cancel
RES-28080180	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 08:54 PM	Completed	—
RES-66839741	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-87447483	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-59754269	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test cleanup
RES-43772778	941234567V	Galle Fort Coastal Solar Hub 12 kWh • Drop Off	Tue, Oct 06, 11:30 AM	Cancelled	Reason: Suhe cleanup

Showing 1-9 of 9 reservations
Rows per page: 10

SOLVANCE • Smart Solar Microgrid Trading

 SOLVANCE

Grid Operator

PM

Priyantha Malagamuwa

OPERATOR102

Logout

Grid Operator Dashboard

Logged in as Priyantha Malagamuwa (OPERATOR102) • Operating hub: Galle Fort Coastal Solar Hub

Reservation Queue
4 Reservations
Pending & approved, all stations

Verified Trades
1 Completed
Verified by QR scan

Battery Capacity
53 Slots Free
Across all active stations

QR Scanner

Reservation Queue (4)

Battery Slots (29 free)

All Panels

Verify QR Pass

Scan the prosumer's QR pass with the webcam, or enter the token manually

Webcam

Manual Entry

Camera Off

Start the webcam to scan the QR pass on the prosumer's phone.

Start Webcam

QR Token

e.g., SOLAR-TX: RES-78442525:87AF828E1B4B or RES-78442525

Verify & Complete

Battery Slots

Update how many battery slots are free at this station

Station

MG-COL-01 — Galle Fort Coastal Solar Hub (29/30 Available)

Occupied Slots

1 / 30

Occupancy

3.3%

Free slots

29 of 30 free

0 (full)

30 (all free)

-1 Slot

29

+1 Slot

Save

Reservation Queue

Upcoming reservations first (soonest at the top), then overdue ones, then history.

+ New Reservation

Filter NIC, Res # or Hub

All (?)

Pending (4)

Approved (0)

Completed (1)

Cancelled (4)

Reservation #	Prosumer NIC	Station & Trade Type	Scheduled Window	Status	Actions
RES-95218332	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Wed, Oct 07, 04:01 AM Editable - 29.6h left	Pending	Approve Edit Cancel
RES-48277478	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 09:19 PM Editable - 166.9h left	Pending	Approve Edit Cancel
RES-11637664	288456556578	Colombo Harbor Microgrid Substation 10 kWh • Charging Other Hub	Mon, Oct 12, 09:26 PM Editable - 167.1h left	Pending	Approve Edit Cancel
RES-37548867	941234567V	Galle Fort Coastal Solar Hub 10 kWh • Drop Off	Mon, Oct 05, 12:00 PM Time passed	Pending	Approve Edit Cancel
RES-28080100	288456556578	Galle Fort Coastal Solar Hub 15 kWh • Drop Off	Mon, Oct 12, 08:54 PM	Completed	—
RES-66839741	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test clean up
RES-87447403	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test clean up
RES-59754269	941234567V	Galle Fort Coastal Solar Hub 20 kWh • Drop Off	Sat, Oct 10, 11:45 PM	Cancelled	Reason: Test clean up
RES-43772778	941234567V	Galle Fort Coastal Solar Hub 12 kWh • Drop Off	Tue, Oct 06, 11:30 AM	Cancelled	Reason: Suite cleanup

Showing 1-9 of 9 reservations

Rows per page: 10

SOLVANCE • Smart Solar Microgrid Trading

Backoffice

Overview
Prosumers
Solar Hubs
Staff & Operators
Reservations

SA System Administrator
200331713189
Logout

Prosumer Directory

Review new registrations, check National Identity Card (NIC) details and manage account status.

Refresh
Add Prosumer

Total Prosumers
4
All account statuses

Pending Approvals
0
Awaiting Backoffice review

Active Prosumers
4
Can login and reserve energy

Deactivated Accounts
0
Only Backoffice can reactivate

All 4 Pending 0 Active 4 Deactivated 0

Prosumer & NIC	Solar Installation	Contact Details	Verification & Status	Actions
Sunil Silva # 941234567Y Male • Born 1994 (32 yrs) Approved by 200331713189	12 kW No inverter serial Kandy Solar Site	sunil.silva@test.lk +94778899001	Active Document Missing Front Back	Review NIC Deactivate
Sakith Chanaka # 20042561173 Female • Born 2004 (22 yrs) Approved by 200331713189	75 kW INV-SL-26-102 synergy maharagama	cryptonkadet@gmail.com 0768286980	Active NIC Verified Front Back	Review NIC Deactivate
Test Prosumer User # 199812345678 Male • Born 1998 (28 yrs) Approved by 200331713189	10.5 kW INV-TEST-981 Galle Grid Station	testprosumer@solvance.lk +94771234567	Active Document Missing Front Back	Review NIC Deactivate
Sakith Liyanage # 20042561173 Male • Born 2004 (22 yrs) Approved by 200331713189	5 kW 1123 65/7, First Lane, Katuwana...	sakithchanaka2004@gmail.com 0769092755	Active NIC Verified Front Back	Review NIC Deactivate

Showing 1-4 of 4 prosumers
Rows per page 10

Smart Solar Microgrid Trading

Backoffice

Overview
Prosumers
Solar Hubs
Staff & Operators
Reservations

SA System Administrator
200331713189
Logout

Prosumer Directory

Review new registrations, check National Identity Card (NIC) details and manage account status.

Refresh
Add Prosumer

Total Prosumers
4
All account statuses

Pending Approvals
0
Awaiting Backoffice review

Active Prosumers
4
Can login and reserve energy

Deactivated Accounts
0
Only Backoffice can reactivate

All 4 Pending 0 Active 4 Deactivated 0

Prosumer & NIC	Solar Installation	Contact Details	Verification & Status	Actions
Sunil Silva # 941234567Y Male • Born 1994 (32 yrs) Approved by 200331713189	12 kW No inverter serial Kandy Solar Site	sunil.silva@test.lk +94778899001	Active Document Missing Front Back	Review NIC Deactivate
Sakith Chanaka # 20042561173 Female • Born 2004 (22 yrs) Approved by 200331713189	75 kW INV-SL-26-102 synergy maharagama	cryptonkadet@gmail.com 0768286980	Active NIC Verified Front Back	Review NIC Deactivate
Test Prosumer User # 199812345678 Male • Born 1998 (28 yrs) Approved by 200331713189	10.5 kW INV-TEST-981 Galle Grid Station	testprosumer@solvance.lk +94771234567	Active Document Missing Front Back	Review NIC Deactivate
Sakith Liyanage # 20042561173 Male • Born 2004 (22 yrs) Approved by 200331713189	5 kW 1123 65/7, First Lane, Katuwana...	sakithchanaka2004@gmail.com 0769092755	Active NIC Verified Front Back	Review NIC Deactivate

Showing 1-4 of 4 prosumers
Rows per page 10

Smart Solar Microgrid Trading

Add Prosumer

New accounts start as **Pending** until a Backoffice officer approves them.

National Identity Card (NIC) *
e.g. 981234567Y or 200812345678

Full Name *
e.g. Sunil Shantha Perera

Email Address *
prosumer@solarhub.lk

Phone Number *
+9477123 4567

Installation Address
No. 45, Solar Avenue, Colombo 03

Solar Capacity (kW) *
e.g. 15 kW

Inverter Serial Number
e.g. INV-HUA-9988-LK

Password *

At least 6 characters. The prosumer uses it to sign-in to the mobile app.

NIC Documents

The front copy is needed for the document check

NIC Front

NIC Back (optional)

Cancel
Create Prosumer

Backoffice

Overview
Prosumers
Solar Hubs
Staff & Operators
Reservations

SA System Administrator
290531733189
Logout

Solar Hubs

Configure capacity, battery storage slots, opening hours and trading slots.

Cards
Map
Refresh
Add Solar Hub

Notice: A hub with upcoming Pending or Approved reservations can't be deactivated or deleted until those reservations are completed or cancelled.

All (2)
Active (2)
Inactive (0)

Galle Fort Coastal Solar Hub
6.0328, 80.2170 • Rampart Street, Galle, Southern Province

Open in Google Maps

MG-COL-01 Active

Galle Fort Coastal Solar Hub
Rampart Street, Galle, Southern Province

1500 kWh • 29/30 battery slots free Trading slots

MG-COL-02 Active

Colombo Harbor Microgrid Substation
Port Access Road, Colombo 13, Western Province

1000 kWh • 24/24 battery slots free Trading slots

Showing 1-2 of 2 hubs

SOLVANCE • Smart Solar Microgrid Trading

6.2.Android Application Screens

9:26

 **SOLVANCE**
Clean Energy Microgrid

Prosumer Onboarding

Register with your National Identity Card (NIC). Account starts in Pending state until verified by backoffice.

National Identity Card (NIC) *

Full Legal Name *

Email Address *

Mobile Phone Number *

Physical / Property Address *

Solar Array (kW) * Inverter Serial

Password (Min 6 chars) *



 **e-KYC Document Verification** Required

Capture or upload official documents to authenticate your identity and solar microgrid connection node.

1. NIC Front Side (Photo & Name) *

No front ID attached

 **Camera**  **Browse Files**

9:19

 **SECURE ENERGY PASS**

Digital Booking Pass

RES-78442525 



SCAN AT SOLAR STATION Kiosk

BOOKING STATUS Approved

 **Colombo Central Solar Hub**

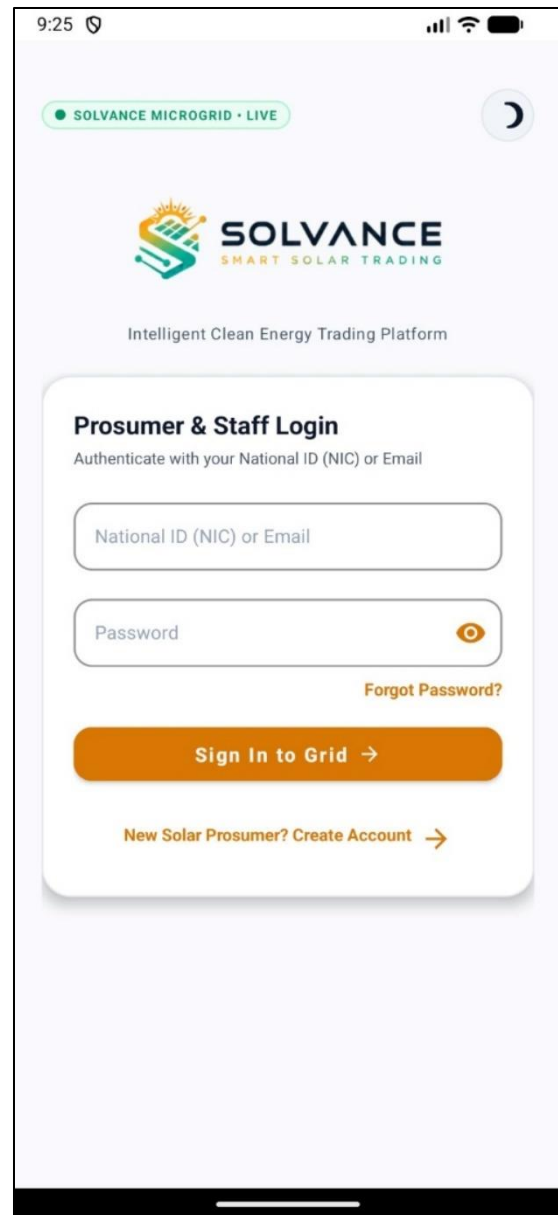
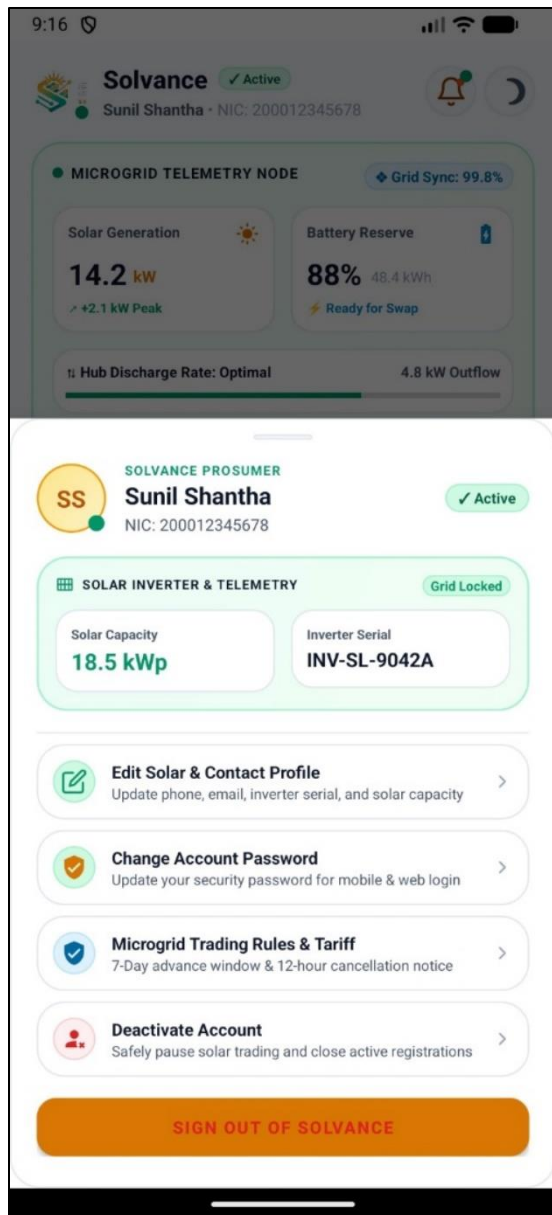
 Scheduled: 2026-09-23T20:20:55Z

 **10.0 kWh (Drop-Off / Sell)**

 **12-Hour Rule:** Modifications and cancellations require at least 12 hours' advance notice per Microgrid policy.

 **Modification Locked (<12h)**

 **Cancel Locked (<12h)**  **Dashboard**



9:12

7-Day Window

Book Power Trading Slot

Reserve battery swapping or solar grid charging. Instant encrypted verification pass.

SELECT SOLAR STATION HUB *

Colombo Central Solar Hub (6 slots..)

ENERGY TRADE TYPE *

Drop-Off (Sell)

Charging (Buy)

ENERGY AMOUNT (KWH) *

Energy Amount (kWh)

15.0

10 kWh

15 kWh

25 kWh

50 kWh

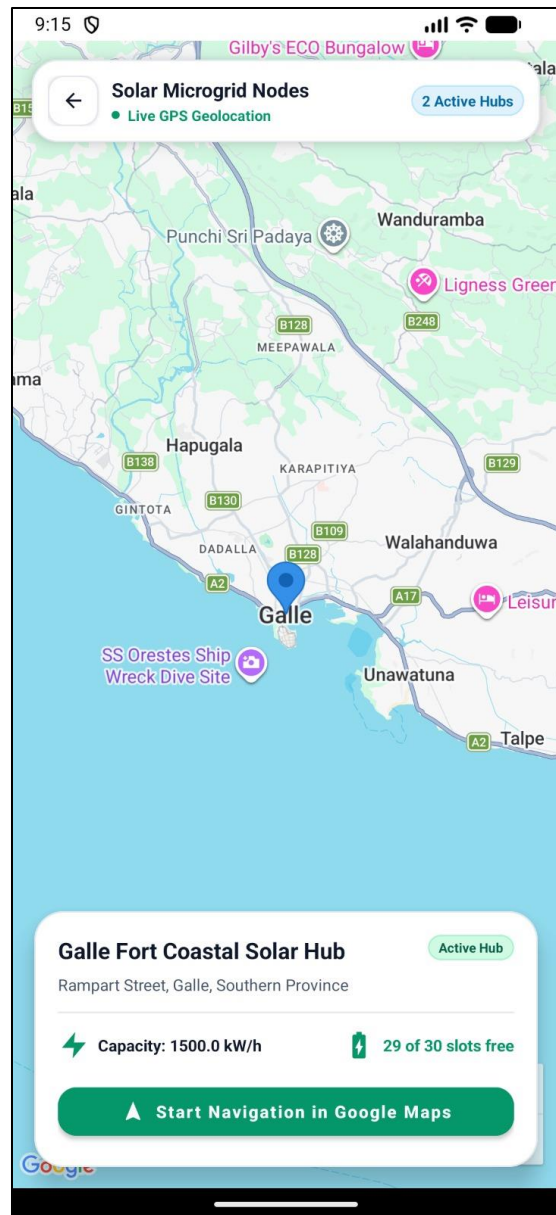
APPOINTMENT SCHEDULE *

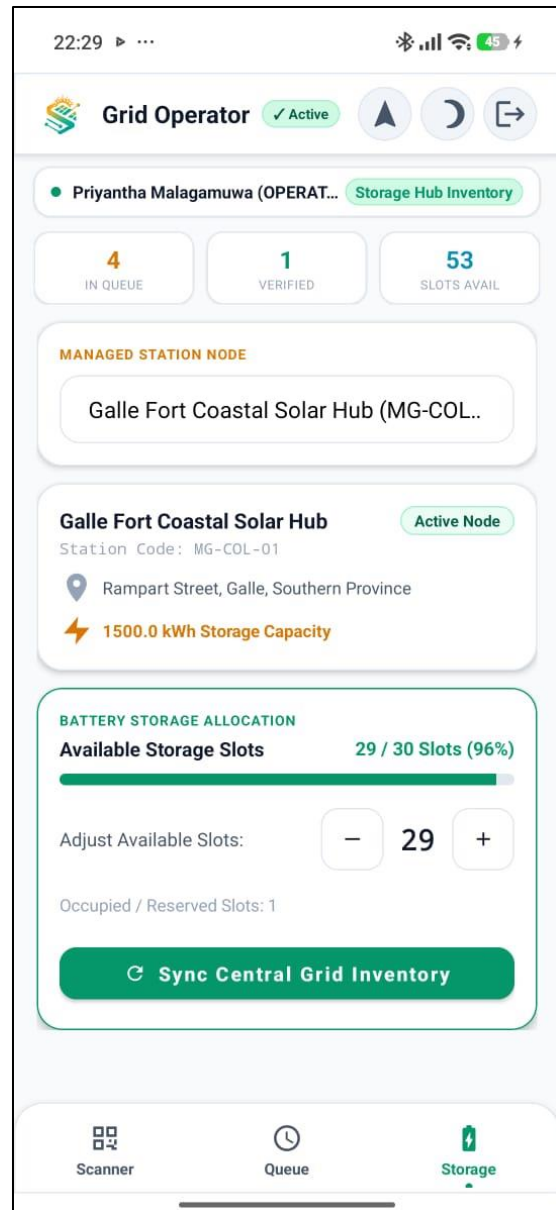
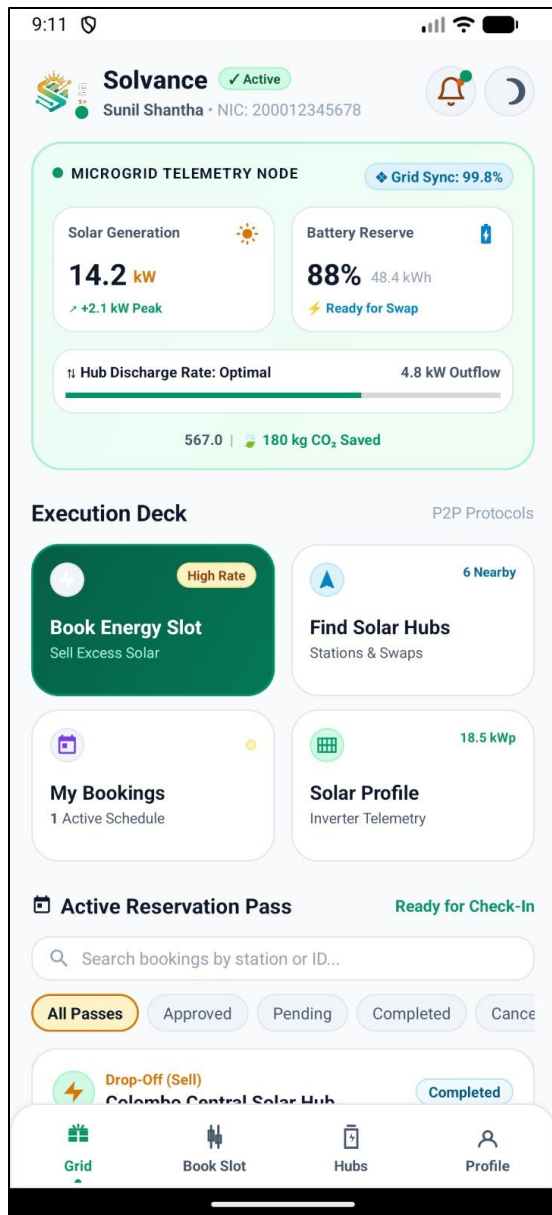
Select Date

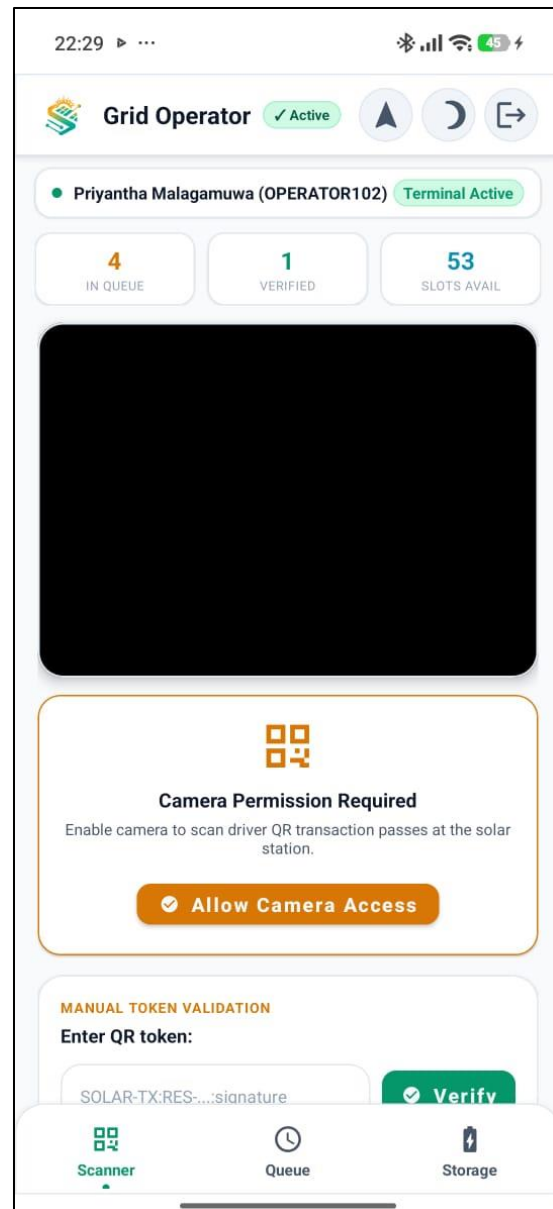
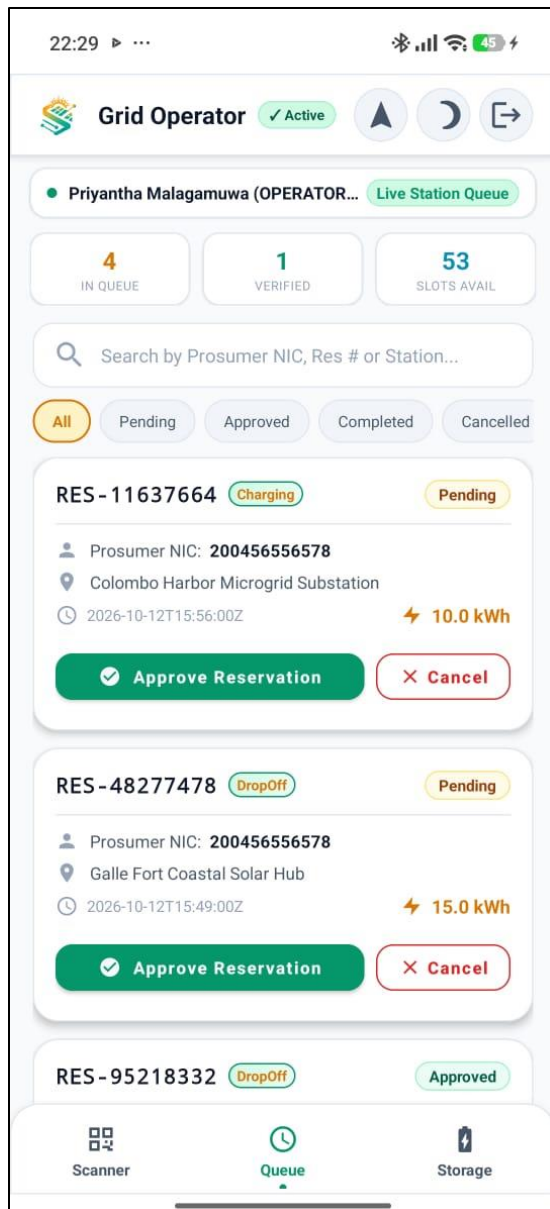
Select Time

Scheduled: Tap Date & Time above

Confirm & Generate QR Pass







7. Source Code / Implementation

7.1.M.L. Booso (IT23452916) - User Identity & Prosumer Lifecycle

Backend - User management

File: `backend/Controllers/UsersController.cs` – `GetPendingProsumers`

```
public async Task<IActionResult> GetPendingProsumers()
{
    // Method: GetPendingProsumers - Gets list of prosumer accounts awaiting approval.
    var prosumers = await _userService.GetPendingProsumersAsync();
    var result = prosumers.Select(u => new
    {
        u.Id,
        u.Nic,
        u.Username,
        u.FullName,
        u.Email,
        u.Phone,
        u.Address,
        u.Role,
        u.Status,
        u.SolarCapacityKw,
        u.InverterSerial,
        u.NicDocumentBase64,
        u.NicBackDocumentBase64,
        u.UtilityBillBase64,
        u.KycTrustScore,
        u.KycRiskLevel,
        u.KycNotes,
        u.RegisteredAt,
        u.CreatedAt
    });
    return Ok(result);
}
```

File: `backend/Controllers/UsersController.cs` – `UpdateStatus`

```
public async Task<IActionResult> UpdateStatus(string nic, [FromBody] UpdateStatusDto dto)
{
    // Method: UpdateStatus - Backoffice action to approve, deactivate, or reactivate user
    accounts.
    var currentRole = User.FindFirstValue(ClaimTypes.Role) ?? "Backoffice";
    var operatorNic = User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "ADMIN001";

    var targetUser = await _userService.GetUserByNicAsync(nic);
    if (targetUser == null)
    {
        return NotFound(new { message = $"User with NIC '{nic}' not found." });
    }

    // Security Rule: Administrators cannot alter status of other Backoffice Administrators
}
```

```

        if (targetUser.Role == "Backoffice" && targetUser.Nic != operatorNic)
        {
            return StatusCode(StatusCodes.Status403Forbidden, new { message = "Security policy
violation: Administrators cannot alter status of other Backoffice Administrators." });
        }

        try
        {
            var success = await _userService.UpdateUserStatusAsync(nic, dto.Status, currentRole,
operatorNic);
            if (!success)
            {
                return NotFound(new { message = $"User with NIC '{nic}' not found." });
            }

            return Ok(new { message = $"Account '{nic}' status successfully changed to
'{dto.Status}'." });
        }
        catch (InvalidOperationException ex)
        {
            return Conflict(new { message = ex.Message });
        }
        catch (UnauthorizedAccessException ex)
        {
            return StatusCode(StatusCodes.Status403Forbidden, new { message = ex.Message });
        }
    }
}

```

File: `backend/Services/UserService.cs` - `RegisterProsumerAsync`

```

public async Task<User> RegisterProsumerAsync(ProsumerRegisterDto dto)
{
    // Method: RegisterProsumerAsync - Creates new Prosumer account with initial "Pending" status
and hardware solar specs.
    var existingUser = await _context.Users.Find(u => u.Nic == dto.Nic || u.Email ==
dto.Email).FirstOrDefaultAsync();
    if (existingUser != null)
    {
        throw new InvalidOperationException("User with this NIC or Email already registered.");
    }

    var prosumer = new User
    {
        Nic = dto.Nic.Trim().ToUpper(),
        Username = dto.Nic.Trim().ToUpper(),
        FullName = dto.FullName.Trim(),
        Email = dto.Email.Trim().ToLower(),
        Phone = dto.Phone.Trim(),
        Address = dto.Address.Trim(),
        SolarCapacityKw = dto.SolarCapacityKw,
        InverterSerial = dto.InverterSerial.Trim(),
        NicDocumentBase64 = dto.NicDocumentBase64,
        NicBackDocumentBase64 = dto.NicBackDocumentBase64,
        UtilityBillBase64 = dto.UtilityBillBase64,
    }
}

```

```

        KycTrustScore = (!string.IsNullOrEmpty(dto.NicDocumentBase64) &&
!string.IsNullOrEmpty(dto.NicBackDocumentBase64)) ? 99 : (!string.IsNullOrEmpty(dto.NicDocumentBase64) ?
95 : 85),

        KycRiskLevel = "Low",
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),
        Role = "Prosumer",
        Status = _securitySettings.DefaultProsumerStatus ?? "Pending",
        RegisteredAt = DateTime.UtcNow,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _context.Users.InsertOneAsync(prosumer);
    return prosumer;
}

```

File: `backend/Services/UserService.cs` - `UpdateUserStatusAsync`

```

public async Task<bool> UpdateUserStatusAsync(string nic, string newStatus, string operatorRole, string?
operatorNic = null)
{
    // Method: UpdateUserStatusAsync - Updates account status (Activate/Deactivate/Reactivate)
    enforcing Backoffice rule and audit fields.
    if (operatorRole != "Backoffice")
    {
        throw new UnauthorizedAccessException("Only Backoffice officers have permission to
activate or reactivate accounts.");
    }

    var cleanNic = nic.Trim().ToUpperInvariant();

    // If deactivating a prosumer, verify no active/pending reservations exist
    if (newStatus == "Deactivated")
    {
        var activeCount = await _context.Reservations.CountDocumentsAsync(r =>
            (r.ProsumerNic == cleanNic || r.ProsumerNic == nic.Trim()) &&
            (r.Status == "Pending" || r.Status == "Approved") &&
            r.ScheduledDateTime >= DateTime.UtcNow
        );

        if (activeCount > 0)
        {
            throw new InvalidOperationException($"Cannot deactivate prosumer '{cleanNic}'. User
has {activeCount} active or pending energy reservations.");
        }
    }

    var updateBuilder = Builders<User>.Update
        .Set(u => u.Status, newStatus)
        .Set(u => u.UpdatedAt, DateTime.UtcNow);

    if (newStatus == "Active")
    {
        updateBuilder = updateBuilder

```

```

        .Set(u => u.ActivatedAt, DateTime.UtcNow)
        .Set(u => u.ApprovedBy, !string.IsNullOrEmpty(operatorNic) ? operatorNic :
"Backoffice"));
    }

    var result = await _context.Users.UpdateOneAsync(u => u.Nic == cleanNic || u.Nic ==
nic.Trim(), updateBuilder);
    return result.ModifiedCount > 0;
}

```

File: `backend/Program.cs` – JWT Bearer authentication and authorization configuration

```

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    // Method: ConfigureJwtBearer - Enforces token signing key, lifetime, and audience validation.
    options.RequireHttpsMetadata = false;
    options.SaveToken = true;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(key),
        ValidateIssuer = true,
        ValidIssuer = !string.IsNullOrEmpty(jwtSettings.Issuer) ? jwtSettings.Issuer :
(builder.Configuration["JwtSettings:Issuer"] ?? "SolarMicrogridApi"),
        ValidateAudience = true,
        ValidAudience = !string.IsNullOrEmpty(jwtSettings.Audience) ? jwtSettings.Audience :
(builder.Configuration["JwtSettings:Audience"] ?? "SolarMicrogridClients"),
        ClockSkew = TimeSpan.Zero
    };
});

builder.Services.AddAuthorization();

var app = builder.Build();

```

Web client - Staff authentication and prosumer management

File: `web-client/src/pages/Login.jsx` – `handleSubmit`

```

const handleSubmit = async (e) => {
    if (e) e.preventDefault();
    if (loading) return;
    setError('');
    setLoading(true);

    try {
        const response = await api.post('/auth/login', {
            usernameOrNic: usernameOrNic.trim(),

```

```

        password: password
    });

    const { token, role, nic, fullName, email, status, mustChangePassword } = response.data;

    // Ensure only web-authorized roles can access the web console
    if (role !== 'Backoffice' && role !== 'GridOperator') {
        setError('This console is for Backoffice and Grid Operator staff. Prosumers sign in with the Solvance Android app.');
```

Solvance Android app.');

```

        setLoading(false);
        return;
    }

    // If administrator required mandatory password change on sign-in, intercept session
    if (mustChangePassword) {
        setPendingAuth({ token, role, nic, fullName, email, status });
        setNewPassword('');
        setConfirmPassword('');
        setChangeError('');
        setMustChangePasswordModal(true);
        setLoading(false);
        return;
    }

    localStorage.setItem('solar_auth_token', token);
    localStorage.setItem('solar_user_data', JSON.stringify({ nic, fullName, email, role, status }));
    onLoginSuccess({ nic, fullName, email, role, status });
} catch (err) {
    // No response at all means the API is unreachable, not that the password is wrong
    if (!err.response) {
        setError('Cannot reach the server. Check your connection and try again.');
```

again.');

```

    } else {
        // The API explains lockouts (423) and pending/deactivated accounts (403) in `message`
        setError(err.response.data?.message || 'Sign-in failed. Please check your details and try
again.');
```

again.');

```

    }
} finally {
    setLoading(false);
}
};

```

File: ``web-client/src/pages/ProsumerManagement.jsx`` – status lifecycle

```

const handleStatusChange = async (nic, newStatus) => {
    if (statusUpdatingNic) return;
    setStatusUpdatingNic(nic);
    setActionError('');
    try {
        await api.put(`/users/${nic}/status`, { status: newStatus });
        notify(`Prosumer ${nic} is now ${newStatus}.`, 'success');
        setShowKycModal(false);
        setConfirmDeactivate(null);
        fetchProsumers();
    } catch (err) {

```

```

    // e.g. 409 when the prosumer still has upcoming bookings; shown in the open modal and on the page
    const errMsg = getApiError(err, 'Failed to update account status.');
```

```

    setActionError(errMsg);
    notify(errMsg, 'error');
  } finally {
    setStatusUpdatingNic(null);
  }
};

```

File: ``web-client/src/pages/ProsumerManagement.jsx`` – prosumer registration and NIC validation

```

const handleCreateProsumer = async (e) => {
  e.preventDefault();
  if (submitting) return;
  setFormError('');
  if (!formData.nic || !formData.fullName.trim() || !formData.email.trim() || !formData.password) {
    setFormError('Please complete all required fields.');
```

```

    return;
  }

  const nicInfo = parseSriLankanNic(formData.nic);
  if (!nicInfo.isValid) {
    setFormError(nicInfo.error || 'Invalid NIC format.');
```

```

    return;
  }

  // Same minimum as the API (ProsumerRegisterDto: MinLength(6))
  if (formData.password.length < 6) {
    setFormError('Password must be at least 6 characters.');
```

```

    return;
  }

  try {
    setSubmitting(true);
    const nic = formData.nic.trim().toUpperCase();

    await api.post('/auth/register-prosumer', {
      nic,
      fullName: formData.fullName.trim(),
      email: formData.email.trim().toLowerCase(),
      phone: formData.phone.trim(),
      address: formData.address.trim(),
      solarCapacityKw: parseFloat(formData.solarCapacityKw) || 0,
      inverterSerial: formData.inverterSerial.trim(),
      password: formData.password,
      nicDocumentBase64: formData.nicDocumentBase64 || null,
      nicBackDocumentBase64: formData.nicBackDocumentBase64 || null,
      utilityBillBase64: formData.utilityBillBase64 || null
    });

    notify(`Prosumer ${nic} created. The account is Pending until a Backoffice officer approves it.`,
    'success');
```

```

    setShowCreateModal(false);
    setFormData(initialForm);
  }
};

```

```

        fetchProsumers();
    } catch (err) {
        setFormError(getApiError(err, 'Failed to register prosumer.));
    } finally {
        setSubmitting(false);
    }
};

```

Android client - NIC validation

File: `mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/auth/RegisterActivity.java` - registration validation

```

String inverterSerial = etInverterSerial.getText() != null ? etInverterSerial.getText().toString().trim()
: "";

String password = etPassword.getText() != null ? etPassword.getText().toString().trim() : "";

// 1. Mandatory Field Validation
if (nic.isEmpty()) {
    Toast.makeText(this, "National Identity Card (NIC) number is required.",
Toast.LENGTH_SHORT).show();
    etNic.requestFocus();
    return;
}

// 2. Sri Lankan NIC Algorithm Check
NicValidator.NicValidationResult nicResult = NicValidator.validate(nic);
if (!nicResult.isValid) {
    Toast.makeText(this, nicResult.error, Toast.LENGTH_LONG).show();
    etNic.requestFocus();
    return;
}

// 3. Name Validation
if (fullName.isEmpty()) {
    Toast.makeText(this, "Full Name is required.", Toast.LENGTH_SHORT).show();
    etFullName.requestFocus();
    return;
}

// 4. Email Validation
if (email.isEmpty()) {
    Toast.makeText(this, "Email address is required.", Toast.LENGTH_SHORT).show();
    etEmail.requestFocus();
    return;
}
if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
    Toast.makeText(this, "Please enter a valid email address.", Toast.LENGTH_SHORT).show();
    etEmail.requestFocus();
    return;
}
}

```

7.2.G.L.S. Chanlaka (IT23151260) - Solar Nodes & Station Geolocation

Backend - Station and slot management

File: ``backend/Controllers/StationsController.cs`` - ``Create``

```
public async Task<IActionResult> Create([FromBody] SolarStationDto dto)
{
    // Method: Create - Registers new solar hub with GPS location, capacity, and operational
    schedule.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var created = await _stationService.CreateStationAsync(dto);
    return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
}
```

File: ``backend/Controllers/StationsController.cs`` - ``Update``

```
public async Task<IActionResult> Update(string id, [FromBody] SolarStationDto dto)
{
    // Method: Update - Modifies station metadata, schedule, and technical specifications.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var success = await _stationService.UpdateStationAsync(id, dto);
    if (!success)
    {
        return NotFound(new { message = "Station not found." });
    }
    return Ok(new { message = "Solar station updated successfully." });
}
```

File: ``backend/Controllers/StationsController.cs`` - ``Deactivate``

```
public async Task<IActionResult> Deactivate(string id)
{
    // Method: Deactivate - Deactivates node ensuring no pending or approved reservations are
    blocked or stranded.
    try
    {
        var success = await _stationService.DeactivateStationAsync(id);
        if (!success)
        {
            return NotFound(new { message = "Station not found." });
        }

        return Ok(new { message = "Solar station deactivated successfully." });
    }
    catch (InvalidOperationException ex)
    {
    }
}
```

```

        return BadRequest(new { message = ex.Message });
    }
}

```

File: `backend/Controllers/StationsController.cs` – `Reactivate`

```

public async Task<IActionResult> Reactivate(string id)
{
    // Method: Reactivate - Restores inactive solar hub to active operational service.
    var success = await _stationService.ReactivateStationAsync(id);
    if (!success)
    {
        return NotFound(new { message = "Station not found." });
    }

    return Ok(new { message = "Solar station reactivated successfully." });
}

```

File: `backend/Services/StationService.cs` – `CreateStationAsync`

```

public async Task<SolarStation> CreateStationAsync(SolarStationDto dto)
{
    // Method: CreateStationAsync - Registers a new solar microgrid hub with GPS coordinates and
    battery specs.
    var station = new SolarStation
    {
        StationCode = dto.StationCode.Trim().ToUpper(),
        Name = dto.Name.Trim(),
        Latitude = dto.Latitude,
        Longitude = dto.Longitude,
        Address = dto.Address.Trim(),
        CapacityKwh = dto.CapacityKwh,
        TotalBatterySlots = dto.TotalBatterySlots,
        AvailableBatterySlots = dto.AvailableBatterySlots > 0 ? dto.AvailableBatterySlots :
dto.TotalBatterySlots,
        GridConnection = !string.IsNullOrEmpty(dto.GridConnection) ? dto.GridConnection :
"Three-Phase 400V AC",
        StorageType = !string.IsNullOrEmpty(dto.StorageType) ? dto.StorageType : "LiFePO4
BESS",
        MaxDischargeRateKw = dto.MaxDischargeRateKw > 0 ? dto.MaxDischargeRateKw : 150.0,
        Schedule = dto.Schedule,
        IsActive = true,
        CreatedAt = DateTime.UtcNow
    };

    await _context.Stations.InsertOneAsync(station);

    if (dto.AutoGenerateSlots && !string.IsNullOrEmpty(station.Id))
    {
        await GenerateHourlySlotsAsync(station.Id, 7);
    }

    return station;
}

```

File: `backend/Services/StationService.cs` - `UpdateStationAsync`

```
public async Task<bool> UpdateStationAsync(string id, SolarStationDto dto)
{
    // Method: UpdateStationAsync - Updates solar hub specifications, operational hours, and
    metadata.
    var update = Builders<SolarStation>.Update
        .Set(s => s.StationCode, dto.StationCode.Trim().ToUpper())
        .Set(s => s.Name, dto.Name.Trim())
        .Set(s => s.Latitude, dto.Latitude)
        .Set(s => s.Longitude, dto.Longitude)
        .Set(s => s.Address, dto.Address.Trim())
        .Set(s => s.CapacityKwh, dto.CapacityKwh)
        .Set(s => s.TotalBatterySlots, dto.TotalBatterySlots)
        .Set(s => s.AvailableBatterySlots, dto.AvailableBatterySlots)
        .Set(s => s.GridConnection, !string.IsNullOrEmpty(dto.GridConnection) ?
dto.GridConnection : "Three-Phase 400V AC")
        .Set(s => s.StorageType, !string.IsNullOrEmpty(dto.StorageType) ? dto.StorageType :
"LiFePO4 BESS")
        .Set(s => s.MaxDischargeRateKw, dto.MaxDischargeRateKw > 0 ? dto.MaxDischargeRateKw :
150.0)
        .Set(s => s.Schedule, dto.Schedule)
        .Set(s => s.IsActive, dto.IsActive);

    var result = await _context.Stations.UpdateOneAsync(s => s.Id == id, update);
    return result.ModifiedCount > 0;
}
```

File: `backend/Services/StationService.cs` - `DeactivateStationAsync`

```
public async Task<bool> DeactivateStationAsync(string id)
{
    // Method: DeactivateStationAsync - Deactivates station while strictly blocking if active
    energy reservations exist.
    var activeReservationsCount = await _context.Reservations.CountDocumentsAsync(r =>
        r.StationId == id &&
        (r.Status == "Pending" || r.Status == "Approved") &&
        r.ScheduledDateTime >= DateTime.UtcNow
    );

    if (activeReservationsCount > 0)
    {
        throw new InvalidOperationException($"Cannot deactivate station '{id}'. There are
{activeReservationsCount} active or pending energy reservations.");
    }

    var update = Builders<SolarStation>.Update.Set(s => s.IsActive, false);
    var result = await _context.Stations.UpdateOneAsync(s => s.Id == id, update);
    return result.ModifiedCount > 0;
}
```

File: `backend/Controllers/SlotsController.cs` - `Create`

```

public async Task<IActionResult> Create([FromBody] EnergySlot slot)
{
    // Method: Create - Inserts a new booking slot into EnergyBookingSlots collection.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    try
    {
        var created = await _slotService.CreateSlotAsync(slot);
        return Created($"{"/api/slots/{created.Id}", created);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

```

File: `backend/Controllers/SlotsController.cs` - `UpdateAvailability`

```

public async Task<IActionResult> UpdateAvailability(string id, [FromBody] UpdateSlotAvailabilityDto dto)
{
    // Method: UpdateAvailability - Modifies slot capacity status and remaining slots.
    var success = await _slotService.UpdateSlotAvailabilityAsync(id, dto.AvailableSlots,
dto.AllocatedKwh);
    if (!success)
    {
        return NotFound(new { message = "Slot not found." });
    }

    return Ok(new { message = "Slot availability updated successfully." });
}

```

File: `backend/Controllers/SlotsController.cs` - `Delete`

```

public async Task<IActionResult> Delete(string id)
{
    // Method: Delete - Deletes trading slot ensuring no active or pending reservations are
    affected.
    try
    {
        var success = await _slotService.DeleteSlotAsync(id);
        if (!success)
        {
            return NotFound(new { message = "Slot not found." });
        }
    }
}

```

```

    }

    return Ok(new { message = "Trading slot deleted successfully." });
}
catch (InvalidOperationException ex)
{
    return BadRequest(new { message = ex.Message });
}
}

```

File: `backend/Services/SlotService.cs` - `CreateSlotAsync`

```

public async Task<EnergySlot> CreateSlotAsync(EnergySlot slot)
{
    // Method: CreateSlotAsync - Creates a new trading slot for prosumer reservations with
    validation and overlap check.
    if (string.IsNullOrEmpty(slot.StationId))
    {
        throw new ArgumentException("Station ID is required.");
    }
    if (string.IsNullOrEmpty(slot.Date))
    {
        throw new ArgumentException("Date (YYYY-MM-DD) is required.");
    }
    if (string.IsNullOrEmpty(slot.StartTime) || string.IsNullOrEmpty(slot.EndTime))
    {
        throw new ArgumentException("Start time and end time are required.");
    }
    if (string.Compare(slot.EndTime, slot.StartTime, StringComparison.Ordinal) <= 0)
    {
        throw new ArgumentException("End time must be strictly after start time.");
    }
    if (slot.SlotCapacityKwh <= 0)
    {
        throw new ArgumentException("Slot capacity must be greater than 0 kWh.");
    }
    if (slot.AvailableSlots <= 0)
    {
        throw new ArgumentException("Available reservation slots must be greater than 0.");
    }

    var station = await _context.Stations.Find(s => s.Id == slot.StationId).FirstOrDefaultAsync();
    if (station == null)
    {
        throw new KeyNotFoundException($"Microgrid solar station '{slot.StationId}' not found.");
    }

    // Enforce hub operational hours if defined
    if (station.Schedule != null && !string.IsNullOrEmpty(station.Schedule.OpenTime) &&
    !string.IsNullOrEmpty(station.Schedule.CloseTime))
    {
        if (string.Compare(slot.StartTime, station.Schedule.OpenTime, StringComparison.Ordinal) <
0 ||

```

```

        string.Compare(slot.EndTime, station.Schedule.CloseTime, StringComparison.Ordinal) >
0)
    {
        throw new InvalidOperationException($"Trading slot hours ({slot.StartTime} -
{slot.EndTime}) must be within hub operating schedule ({station.Schedule.OpenTime} -
{station.Schedule.CloseTime}).");
    }
}

// Backend Overlap Protection: check if any existing slot for this station and date collides
with the time interval
var clash = await _context.Slots.Find(s =>
    s.StationId == slot.StationId &&
    s.Date == slot.Date &&
    string.Compare(s.StartTime, slot.EndTime) < 0 &&
    string.Compare(s.EndTime, slot.StartTime) > 0
).FirstOrDefaultAsync();

if (clash != null)
{
    throw new InvalidOperationException($"An overlapping trading slot ({clash.StartTime} -
{clash.EndTime}) already exists on {slot.Date}.");
}

slot.CreatedAt = DateTime.UtcNow;
if (string.IsNullOrEmpty(slot.Status))
{
    slot.Status = "Open";
}

await _context.Slots.InsertOneAsync(slot);
return slot;
}

```

File: `backend/Services/SlotService.cs` - `UpdateSlotAvailabilityAsync`

```

public async Task<bool> UpdateSlotAvailabilityAsync(string slotId, int availableSlots, double
allocatedKwh)
{
    // Method: UpdateSlotAvailabilityAsync - Adjusts available slot count and updates status when
capacity reached.
    var existing = await _context.Slots.Find(s => s.Id == slotId).FirstOrDefaultAsync();
    if (existing == null)
    {
        return false;
    }

    var status = availableSlots <= 0 ? "Full" : (existing.Status == "Full" ? "Open" :
existing.Status);
    var update = Builders<EnergySlot>.Update
        .Set(s => s.AvailableSlots, availableSlots)
        .Set(s => s.AllocatedKwh, allocatedKwh)
        .Set(s => s.Status, status);
}

```

```

        var result = await _context.Slots.UpdateOneAsync(s => s.Id == slotId, update);
        return result.MatchedCount > 0;
    }

```

File: `backend/Services/SlotService.cs` – `DeleteSlotAsync`

```

public async Task<bool> DeleteSlotAsync(string id)
{
    // Method: DeleteSlotAsync - Deletes an energy trading slot, blocking if active or pending
    reservations exist.
    var activeReservationsCount = await _context.Reservations.CountDocumentsAsync(r =>
        r.SlotId == id &&
        (r.Status == "Pending" || r.Status == "Approved") &&
        r.ScheduledDateTime >= DateTime.UtcNow
    );

    if (activeReservationsCount > 0)
    {
        throw new InvalidOperationException($"Cannot delete slot '{id}'. There are
{activeReservationsCount} active or pending energy reservations linked to it.");
    }

    var result = await _context.Slots.DeleteOneAsync(s => s.Id == id);
    return result.DeletedCount > 0;
}

```

Web client - Node management and slot creation

File: `web-client/src/pages/NodeManagement.jsx` – station validation and create/update flow

```

const handleSubmit = async (e) => {
    e.preventDefault();
    if (saving) return;
    setFormError('');

    const stationCode = formData.stationCode.trim().toUpperCase();
    const name = formData.name.trim();
    const address = formData.address.trim();
    const latitude = parseFloat(formData.latitude);
    const longitude = parseFloat(formData.longitude);
    const capacityKwh = parseFloat(formData.capacityKwh);
    const totalBatterySlots = parseInt(formData.totalBatterySlots, 10);
    const maxDischargeRateKw = parseFloat(formData.maxDischargeRateKw) || Math.round(capacityKwh * 0.2);

    if (isNaN(latitude) || isNaN(longitude)) {
        setFormError('Please provide valid GPS latitude and longitude coordinates.');
```

return;

```

    }

    if (latitude < 5.8 || latitude > 9.9 || longitude < 79.5 || longitude > 82.0) {
        setFormError('GPS coordinates must be located within Sri Lanka boundaries (Lat 5.9°-9.9°N, Long
79.5°-81.9°E).');
```

return;

```

    }
}

```

```

if (isNaN(capacityKwh) || capacityKwh <= 0) {
  setFormError('Storage capacity must be a positive number in kWh.');
```

```

  return;
}

if (isNaN(totalBatterySlots) || totalBatterySlots < 1) {
  setFormError('Total battery slots must be at least 1.');
```

```

  return;
}

if (toMinutes(formData.closeTime) <= toMinutes(formData.openTime)) {
  setFormError('Closing time must be after opening time.');
```

```

  return;
}

if (!formData.daysOpen || formData.daysOpen.length === 0) {
  setFormError('At least one operational day must be selected.');
```

```

  return;
}

const duplicate = stations.find(
  (s) => s.id !== editingStation?.id && (s.stationCode || '').toUpperCase() === stationCode
);
if (duplicate) {
  setFormError(`Station code ${stationCode} is already used by ${duplicate.name}.`);
  return;
}

if (totalBatterySlots < occupiedSlots) {
  setFormError(`${occupiedSlots} battery slots are occupied right now, so the total can't be lower
than ${occupiedSlots}.`);
  return;
}

if (editingStation) {
  // The backend answers "Station not found" when an update changes nothing, so skip no-op saves
  const unchanged =
    stationCode === editingStation.stationCode &&
    name === editingStation.name &&
    address === editingStation.address &&
    latitude === editingStation.latitude &&
    longitude === editingStation.longitude &&
    capacityKwh === editingStation.capacityKwh &&
    totalBatterySlots === editingStation.totalBatterySlots &&
    formData.openTime === editingStation.schedule?.openTime &&
    formData.closeTime === editingStation.schedule?.closeTime &&
    JSON.stringify(formData.daysOpen) === JSON.stringify(editingStation.schedule?.daysOpen);
  if (unchanged) {
    setIsModalOpen(false);
    showToast('No changes to save.', 'info');
```

```

    return;
  }
}
}

```

```

const payload = {
  stationCode,
  name,
  latitude,
  longitude,
  address,
  capacityKwh,
  totalBatterySlots,
  // Racks that are occupied stay occupied when the total changes
  availableBatterySlots: totalBatterySlots - occupiedSlots,
  schedule: {
    openTime: formData.openTime,
    closeTime: formData.closeTime,
    daysOpen: formData.daysOpen
  },
  autoGenerateSlots: !editingStation && formData.autoGenerateSlots,
  // Editing must not silently reactivate a deactivated hub
  isActive: editingStation ? editingStation.isActive : true
};

setSaving(true);
try {
  if (editingStation) {
    await api.put(`/stations/${editingStation.id}`, payload);
    showToast(`Solar hub '${name}' updated.`);
  } else {
    await api.post('/stations', payload);
    showToast(`Solar hub '${name}' added with 7-day slots generated.`);
  }
  setIsModalOpen(false);
  fetchStations();
} catch (err) {
  setFormError(getApiError(err, 'Failed to save the solar hub.'));
} finally {
  setSaving(false);
}
};

```

File: `web-client/src/pages/NodeManagement.jsx` – deactivate/reactivate/delete actions

```

const runStationAction = async (station, action) => {
  setBusyStationId(station.id);
  try {
    if (action === 'deactivate') {
      await api.post(`/stations/${station.id}/deactivate`);
      showToast(`Solar hub '${station.name}' deactivated.`);
    } else if (action === 'reactivate') {
      await api.post(`/stations/${station.id}/reactivate`);
      showToast(`Solar hub '${station.name}' reactivated.`);
    } else {
      await api.delete(`/stations/${station.id}`);
      showToast(`Solar hub '${station.name}' and its trading slots deleted.`);
    }
  }
}

```

```

    // Keep the card's buttons disabled until it shows the new state
    await fetchStations();
  } catch (err) {
    showToast(getApiError(err, 'Action failed.'), 'error');
  } finally {
    setBusyStationId(null);
    setConfirmAction(null);
  }
};

```

File: ``web-client/src/pages/NodeManagement.jsx`` – slot creation and overlap validation

```

const handleCreateSlot = async (e) => {
  e.preventDefault();
  if (!slotStation || slotSubmitting) return;
  setSlotError('');
  setSlotSuccess('');

  const { date, startTime, endTime } = newSlotForm;
  const start = toMinutes(startTime);
  const end = toMinutes(endTime);
  if (end <= start) {
    setSlotError('End time must be after the start time.');
```

return;

```

  }

  const openTime = slotStation.schedule?.openTime;
  const closeTime = slotStation.schedule?.closeTime;
  if (openTime && closeTime && toMinutes(closeTime) > toMinutes(openTime)
    && (start < toMinutes(openTime) || end > toMinutes(closeTime))) {
    setSlotError(`Slots must fall within the hub's opening hours (${openTime} - ${closeTime}).`);
    return;
  }

  // Checked against the loaded list; with a date filter active that is only the filtered date
  const clash = stationSlots.find(
    (s) => s.date === date && start < toMinutes(s.endTime) && toMinutes(s.startTime) < end
  );
  if (clash) {
    setSlotError(`This overlaps an existing slot on ${formatSlotDate(date)} (${clash.startTime} -
    ${clash.endTime}).`);
    return;
  }

  setSlotSubmitting(true);
  try {
    const payload = {
      stationId: slotStation.id,
      date,
      startTime,
      endTime,
      slotCapacityKwh: parseFloat(newSlotForm.slotCapacityKwh),
      allocatedKwh: 0,
      availableSlots: parseInt(newSlotForm.availableSlots, 10),

```

```

        status: newSlotForm.status || 'Open'
    };
    await api.post('/slots', payload);
    setSlotSuccess(`Trading slot added for ${formatSlotDate(date)}, ${startTime} - ${endTime}.`);
    // Wait for the refreshed list so the overlap check sees the new slot before the button re-enables
    await fetchSlots(slotStation.id, slotDateFilter);
} catch (err) {
    setSlotError(getApiError(err, 'Failed to create energy slot.'));
} finally {
    setSlotSubmitting(false);
}
};

```

Android client - station geolocation

File: `mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/prosumer/StationsMapActivity.java` - cached/live station markers

```

private void loadStationMarkers() {
    // 1. Load from local SQLite cache first (offline resilience)
    List<SolarStation> cached = dbHelper.getCachedStations();
    if (!cached.isEmpty()) {
        cachedStationList = cached;
        updateCountBadge(cachedStationList.size());
        plotGoogleMapStations(cachedStationList);
    }

    // 2. Fetch live station coordinates from API
    ApiClient.getService(this).getStations(true).enqueue(new Callback<List<SolarStation>>() {
        @Override
        public void onResponse(Call<List<SolarStation>> call, Response<List<SolarStation>> response) {
            if (response.isSuccessful() && response.body() != null) {
                cachedStationList = response.body();
                updateCountBadge(cachedStationList.size());
                dbHelper.cacheStations(cachedStationList);
                plotGoogleMapStations(cachedStationList);
            }
        }

        @Override
        public void onFailure(Call<List<SolarStation>> call, Throwable t) {
            Toast.makeText(StationsMapActivity.this, "Displaying offline cached solar stations.",
                Toast.LENGTH_SHORT).show();
        }
    });
}

```

7.3.L.T. Jayawardhana (IT23156760) - Energy Reservations & Rule Engine

Backend - Reservation lifecycle and business rules

File: `backend/Services/ReservationService.cs` - `CreateReservationAsync`

```

public async Task<EnergyReservation> CreateReservationAsync(CreateReservationDto dto)
{
    // Method: CreateReservationAsync - Validates 7-day scheduling constraint, verifies
    // prosumer/station state, and creates booking.
    var now = DateTime.UtcNow;
    var scheduledUtc = dto.ScheduledDateTime.ToUniversalTime();
    var maxDays = _settings.MaxAdvanceBookingDays > 0 ? _settings.MaxAdvanceBookingDays : 7;
    var graceMinutes = _settings.GracePeriodMinutes >= 0 ? _settings.GracePeriodMinutes : 10;

    // Business Rule: Scheduled within configurable advance days (default: 7 days, up to end of
    // 7th calendar day, Sri Lanka time)
    var maxBookingWindowUtc = GetBookingWindowEndUtc(maxDays);
    if (scheduledUtc < now.AddMinutes(-graceMinutes) || scheduledUtc >= maxBookingWindowUtc)
    {
        throw new ArgumentException($"Power trading reservations must be scheduled within
        {maxDays} days from today.");
    }

    // Verify prosumer is active
    var prosumer = await _context.Users.Find(u => u.Nic == dto.ProsumerNic).FirstOrDefaultAsync();
    if (prosumer == null)
    {
        throw new KeyNotFoundException("Prosumer profile not found.");
    }
    if (prosumer.Status != "Active")
    {
        throw new InvalidOperationException($"Cannot create reservation. Account status is
        '{prosumer.Status}'. Only Active accounts can book.");
    }

    // Verify solar hub is active
    var station = await _context.Stations.Find(s => s.Id == dto.StationId).FirstOrDefaultAsync();
    if (station == null)
    {
        throw new KeyNotFoundException("Microgrid solar station not found.");
    }
    if (!station.IsActive)
    {
        throw new InvalidOperationException("The requested solar hub is currently deactivated.");
    }

    // Enforce station capacity business rule: DropOff requires open battery slot
    if (dto.TradeType == "DropOff" && station.AvailableBatterySlots <= 0)
    {
        throw new InvalidOperationException($"The solar hub '{station.Name}' is currently at
        capacity with 0 available battery slots.");
    }

    if (dto.EnergyAmountKwh <= 0)
    {
        throw new ArgumentException("Energy quota must be greater than 0 kWh.");
    }

    if (dto.EnergyAmountKwh > station.CapacityKwh)

```

```

    {
        throw new ArgumentException($"Requested energy quota ({dto.EnergyAmountKwh} kWh) exceeds
station total capacity ({station.CapacityKwh} kWh).");
    }

    var slotReserved = false;
    var slotId = dto.SlotId?.Trim() ?? string.Empty;

    var localScheduled = TimeZoneInfo.ConvertTimeFromUtc(scheduledUtc, BookingTimeZone);
    var dateStr = localScheduled.ToString("yyyy-MM-dd");
    var timeStr = localScheduled.ToString("HH:mm");

    // If slotId is omitted, auto-discover matching slot for this hub by date and hour window
    if (string.IsNullOrEmpty(slotId))
    {
        var matchingSlot = await _context.Slots.Find(s =>
            s.StationId == station.Id &&
            s.Date == dateStr &&
            string.Compare(s.StartTime, timeStr) <= 0 &&
            string.Compare(s.EndTime, timeStr) > 0 &&
            s.AvailableSlots > 0 &&
            s.Status != "Maintenance"
        ).FirstOrDefaultAsync();

        if (matchingSlot != null)
        {
            slotId = matchingSlot.Id ?? string.Empty;
        }
    }

    if (!string.IsNullOrEmpty(slotId))
    {
        var targetSlot = await _context.Slots.Find(s => s.Id == slotId && s.StationId ==
station.Id).FirstOrDefaultAsync();
        if (targetSlot != null)
        {
            if (targetSlot.Status == "Maintenance")
            {
                throw new InvalidOperationException("The requested energy trading slot is
currently undergoing maintenance.");
            }
            if (targetSlot.AvailableSlots <= 0)
            {
                throw new InvalidOperationException("The requested energy trading slot is fully
booked.");
            }
            if (targetSlot.SlotCapacityKwh > 0 && (targetSlot.AllocatedKwh + dto.EnergyAmountKwh)
> targetSlot.SlotCapacityKwh)
            {
                var remainingKwh = Math.Max(0, targetSlot.SlotCapacityKwh -
targetSlot.AllocatedKwh);
                throw new InvalidOperationException($"Requested energy quota
({dto.EnergyAmountKwh} kWh) exceeds remaining capacity ({remainingKwh:F1} kWh) in this trading slot.");
            }
        }
    }

```

```

        var newAvailable = targetSlot.AvailableSlots - 1;
        var newStatus = newAvailable <= 0 ? "Full" : "Open";

        var slotFilter = Builders<EnergySlot>.Filter.And(
            Builders<EnergySlot>.Filter.Eq(s => s.Id, slotId),
            Builders<EnergySlot>.Filter.Eq(s => s.StationId, station.Id),
            Builders<EnergySlot>.Filter.Gt(s => s.AvailableSlots, 0),
            Builders<EnergySlot>.Filter.Ne(s => s.Status, "Maintenance"));
        var slotUpdate = Builders<EnergySlot>.Update
            .Inc(s => s.AvailableSlots, -1)
            .Inc(s => s.AllocatedKwh, dto.EnergyAmountKwh)
            .Set(s => s.Status, newStatus);
        var slotResult = await _context.Slots.UpdateOneAsync(slotFilter, slotUpdate);
        if (slotResult.ModifiedCount == 1)
        {
            slotReserved = true;
        }
    }
}

var reservation = new EnergyReservation
{
    ReservationNumber = $"RES-{Random.Shared.Next(10000000, 99999999)}",
    ProsumerNic = dto.ProsumerNic,
    StationId = station.Id!,
    StationName = station.Name,
    SlotId = slotId,
    ScheduledDateTime = scheduledUtc,
    EnergyAmountKwh = dto.EnergyAmountKwh,
    TradeType = dto.TradeType,
    Status = !string.IsNullOrEmpty(_settings.DefaultBookingStatus) ?
_settings.DefaultBookingStatus : "Pending",
    QrCodeToken = string.Empty,
    SlotReserved = slotReserved,
    CreatedAt = DateTime.UtcNow,
    UpdatedAt = DateTime.UtcNow
};

try
{
    await _context.Reservations.InsertOneAsync(reservation);
}
catch
{
    if (slotReserved && !string.IsNullOrEmpty(slotId))
    {
        await _context.Slots.UpdateOneAsync(
            s => s.Id == slotId && s.StationId == station.Id,
            Builders<EnergySlot>.Update
                .Inc(s => s.AvailableSlots, 1)
                .Inc(s => s.AllocatedKwh, -dto.EnergyAmountKwh));
    }
    throw;
}

```

```

    }
    return reservation;
}

```

File: `backend/Services/ReservationService.cs` – `UpdateReservationAsync`

```

public async Task<EnergyReservation> UpdateReservationAsync(string id, string requestingNic,
UpdateReservationDto dto)
{
    // Method: UpdateReservationAsync - Enforces 12-hour modification rule and validates updated
    schedule within 7 days.
    var reservation = await
_context.Reservations.Find(BuildIdOrResNumberFilter(id)).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new KeyNotFoundException("Reservation not found.");
    }

    // Prosumer ownership check
    if (!string.IsNullOrEmpty(requestingNic) && reservation.ProsumerNic != requestingNic)
    {
        throw new UnauthorizedAccessException("Unauthorized to modify another user's
reservation.");
    }

    if (reservation.Status == "Cancelled" || reservation.Status == "Completed")
    {
        throw new InvalidOperationException($"Cannot modify a reservation that is already
{reservation.Status}.");
    }

    var modNoticeHours = _settings.ModificationNoticeHours > 0 ? _settings.ModificationNoticeHours
: 12;

    var maxDays = _settings.MaxAdvanceBookingDays > 0 ? _settings.MaxAdvanceBookingDays : 7;
    var graceMinutes = _settings.GracePeriodMinutes >= 0 ? _settings.GracePeriodMinutes : 10;
    var scheduledUtc = dto.ScheduledDateTime.ToUniversalTime();
    var resScheduledUtc = reservation.ScheduledDateTime.ToUniversalTime();

    // Business Rule: Updates require at least configurable hours notice (default: 12h) prior to
    original scheduled time
    var hoursNotice = (resScheduledUtc - DateTime.UtcNow).TotalHours;
    if (hoursNotice < modNoticeHours)
    {
        throw new InvalidOperationException($"Modifications require at least {modNoticeHours}
hours' notice. Only {hoursNotice:F1} hours remain before scheduled time.");
    }

    // Business Rule: Updated schedule must also be within the 7-day booking window
    var now = DateTime.UtcNow;
    var maxModWindowUtc = GetBookingWindowEndUtc(maxDays);
    if (scheduledUtc < now.AddMinutes(-graceMinutes) || scheduledUtc >= maxModWindowUtc)
    {
        throw new ArgumentException($"Updated reservation must be scheduled within {maxDays} days
from today.");
    }
}

```

```

    }

    // Business Rule: the edited booking must pass the same hub checks as a new booking
    if (dto.EnergyAmountKwh <= 0)
    {
        throw new ArgumentException("Energy quota must be greater than 0 kWh.");
    }

    var station = await _context.Stations.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
    if (station == null)
    {
        throw new KeyNotFoundException("Microgrid solar station not found.");
    }
    if (!station.IsActive)
    {
        throw new InvalidOperationException("The requested solar hub is currently deactivated.");
    }
    if (dto.EnergyAmountKwh > station.CapacityKwh)
    {
        throw new ArgumentException($"Requested energy quota ({dto.EnergyAmountKwh} kWh) exceeds
station total capacity ({station.CapacityKwh} kWh).");
    }

    // Switching to a drop-off needs a free battery slot (an approved booking must also respect
slots promised to others)
    if (dto.TradeType == "DropOff" && reservation.TradeType != "DropOff")
    {
        if (reservation.Status == "Approved")
        {
            await EnsureDropOffSlotAvailableAsync(station, reservation.Id);
        }
        else if (station.AvailableBatterySlots <= 0)
        {
            throw new InvalidOperationException($"The solar hub '{station.Name}' is currently at
capacity with 0 available battery slots.");
        }
    }

    var originalEnergyKwh = reservation.EnergyAmountKwh;
    var update = Builders<EnergyReservation>.Update
        .Set(r => r.ScheduledDateTime, scheduledUtc)
        .Set(r => r.EnergyAmountKwh, dto.EnergyAmountKwh)
        .Set(r => r.TradeType, dto.TradeType)
        .Set(r => r.UpdatedAt, DateTime.UtcNow);

    reservation.ScheduledDateTime = scheduledUtc;
    reservation.EnergyAmountKwh = dto.EnergyAmountKwh;
    reservation.TradeType = dto.TradeType;

    // Only an approved booking has a QR pass. Its signature covers the schedule and energy, so it
must be
    // re-issued (the prosumer's app shows the new pass after a refresh). Pending bookings get a
token on approval.

```

```

    if (reservation.Status == "Approved")
    {
        var newQr = GenerateSecureQrToken(reservation);
        update = update.Set(r => r.QrCodeToken, newQr);
        reservation.QrCodeToken = newQr;
    }

    await _context.Reservations.UpdateOneAsync(r => r.Id == reservation.Id, update);

    // Check if slot migration is needed
    var targetSlotId = dto.SlotId?.Trim();
    if (string.IsNullOrEmpty(targetSlotId) && scheduledUtc != resScheduledUtc)
    {
        var localNew = TimeZoneInfo.ConvertTimeFromUtc(scheduledUtc, BookingTimeZone);
        var newDateStr = localNew.ToString("yyyy-MM-dd");
        var newTimeStr = localNew.ToString("HH:mm");
        var autoSlot = await _context.Slots.Find(s =>
            s.StationId == reservation.StationId &&
            s.Date == newDateStr &&
            string.Compare(s.StartTime, newTimeStr) <= 0 &&
            string.Compare(s.EndTime, newTimeStr) > 0 &&
            s.AvailableSlots > 0 &&
            s.Status != "Maintenance"
        ).FirstOrDefaultAsync();
        if (autoSlot != null)
        {
            targetSlotId = autoSlot.Id;
        }
    }

    var slotChanged = !string.IsNullOrEmpty(targetSlotId) && targetSlotId != reservation.SlotId;

    if (slotChanged)
    {
        var newSlot = await _context.Slots.Find(s => s.Id == targetSlotId && s.StationId ==
reservation.StationId).FirstOrDefaultAsync();
        if (newSlot == null)
        {
            throw new KeyNotFoundException("Target energy trading slot not found.");
        }
        if (newSlot.Status == "Maintenance")
        {
            throw new InvalidOperationException("Target energy trading slot is under
maintenance.");
        }
        if (newSlot.AvailableSlots <= 0)
        {
            throw new InvalidOperationException("Target energy trading slot is fully booked.");
        }
        if (newSlot.SlotCapacityKwh > 0 && (newSlot.AllocatedKwh + dto.EnergyAmountKwh) >
newSlot.SlotCapacityKwh)
        {
            var rem = Math.Max(0, newSlot.SlotCapacityKwh - newSlot.AllocatedKwh);

```

```

        throw new InvalidOperationException($"Requested energy quota exceeds target slot
remaining capacity ({rem:F1} kWh).");
    }

    // Release previous slot if reserved
    if (reservation.SlotReserved && !string.IsNullOrEmpty(reservation.SlotId))
    {
        await ReleaseSlotAsync(reservation);
    }

    // Reserve new slot
    var newAvail = newSlot.AvailableSlots - 1;
    var newStat = newAvail <= 0 ? "Full" : "Open";
    await _context.Slots.UpdateOneAsync(
        s => s.Id == newSlot.Id,
        Builders<EnergySlot>.Update
            .Inc(s => s.AvailableSlots, -1)
            .Inc(s => s.AllocatedKwh, dto.EnergyAmountKwh)
            .Set(s => s.Status, newStat));

    reservation.SlotId = newSlot.Id!;
    reservation.SlotReserved = true;
    update = update.Set(r => r.SlotId, newSlot.Id!).Set(r => r.SlotReserved, true);
}
else if (reservation.SlotReserved && !string.IsNullOrEmpty(reservation.SlotId))
{
    // Slot didn't change: update energy delta
    var energyDeltaKwh = dto.EnergyAmountKwh - originalEnergyKwh;
    if (energyDeltaKwh != 0)
    {
        await _context.Slots.UpdateOneAsync(
            s => s.Id == reservation.SlotId && s.StationId == reservation.StationId,
            Builders<EnergySlot>.Update.Inc(s => s.AllocatedKwh, energyDeltaKwh));
    }
}

return reservation;
}

```

File: `backend/Services/ReservationService.cs` – `CancelReservationAsync`

```

public async Task<bool> CancelReservationAsync(string id, string requestingNic, string userRole, string
reason)
{
    // Method: CancelReservationAsync - Enforces 12-hour cancellation notice for prosumers, with
override permitted for grid operators.
    var reservation = await
_context.Reservations.Find(BuildIdOrResNumberFilter(id)).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new KeyNotFoundException("Reservation not found.");
    }

    if (reservation.Status == "Cancelled" || reservation.Status == "Completed")

```

```

        {
            throw new InvalidOperationException($"Reservation is already {reservation.Status}.");
        }

        var cancelNoticeHours = _settings.CancellationNoticeHours > 0 ?
_settings.CancellationNoticeHours : 12;
        var resScheduledUtc = reservation.ScheduledDateTime.ToUniversalTime();

        // If requested by prosumer, enforce strict 12-hour rule
        if (userRole == "Prosumer" || (!string.IsNullOrEmpty(requestingNic) && userRole !=
"GridOperator" && userRole != "Backoffice"))
        {
            var hoursNotice = (resScheduledUtc - DateTime.UtcNow).TotalHours;
            if (hoursNotice < cancelNoticeHours)
            {
                throw new InvalidOperationException($"Cancellations require at least
{cancelNoticeHours} hours' notice. Only {hoursNotice:F1} hours remain before scheduled time.");
            }
        }

        var update = Builders<EnergyReservation>.Update
            .Set(r => r.Status, "Cancelled")
            .Set(r => r.CancellationReason, reason)
            .Set(r => r.SlotReserved, false)
            .Set(r => r.UpdatedAt, DateTime.UtcNow);

        var result = await _context.Reservations.UpdateOneAsync(r => r.Id == reservation.Id, update);
        if (result.ModifiedCount > 0)
        {
            if (reservation.SlotReserved && !string.IsNullOrEmpty(reservation.SlotId))
            {
                await ReleaseSlotAsync(reservation);
            }
            return true;
        }
        return false;
    }
}

```

File: `backend/Services/ReservationService.cs` – `ApproveReservationAsync`

```

public async Task<EnergyReservation> ApproveReservationAsync(string id)
{
    // Method: ApproveReservationAsync - Transitions pending reservation to approved and issues QR
    code.

    var reservation = await
_context.Reservations.Find(BuildIdOrResNumberFilter(id)).FirstOrDefaultAsync();
    if (reservation == null)
    {
        throw new KeyNotFoundException("Reservation not found.");
    }

    if (!string.Equals(reservation.Status, "Pending", StringComparison.OrdinalIgnoreCase))
    {
        throw new InvalidOperationException(

```

```

                $"Only pending reservations can be approved. Current status is
'{reservation.Status}'.");
            }

            // Business Rule: an appointment that has already gone by can only be cancelled, not approved
            var graceMinutes = _settings.GracePeriodMinutes >= 0 ? _settings.GracePeriodMinutes : 10;
            if (reservation.ScheduledDateTime.ToUniversalTime() < DateTime.UtcNow.AddMinutes(-
graceMinutes))
            {
                throw new InvalidOperationException("This booking's scheduled time has already passed.
Cancel it instead of approving it.");
            }

            // Re-check the prosumer and the hub: either may have changed since the booking was made
            var prosumer = await _context.Users.Find(u => u.Nic ==
reservation.ProsumerNic).FirstOrDefaultAsync();
            if (prosumer == null || prosumer.Status != "Active")
            {
                throw new InvalidOperationException(
                    $"Cannot approve: the prosumer account is '{prosumer?.Status ?? "missing"}'. Only
Active accounts can trade energy.");
            }

            var station = await _context.Stations.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
            if (station == null)
            {
                throw new KeyNotFoundException("Microgrid solar station not found.");
            }
            if (!station.IsActive)
            {
                throw new InvalidOperationException($"Cannot approve: the solar hub '{station.Name}' is
deactivated.");
            }
            if (reservation.TradeType == "DropOff")
            {
                await EnsureDropOffSlotAvailableAsync(station, reservation.Id);
            }
            var qrToken = GenerateSecureQrToken(reservation);
            var update = Builders<EnergyReservation>.Update
                .Set(r => r.Status, "Approved")
                .Set(r => r.QrCodeToken, qrToken)
                .Set(r => r.UpdatedAt, DateTime.UtcNow);

            await _context.Reservations.UpdateOneAsync(r => r.Id == reservation.Id, update);
            reservation.Status = "Approved";
            reservation.QrCodeToken = qrToken;
            return reservation;
        }
    }

```

File: `backend/Controllers/ReservationsController.cs` – `Create`

```

public async Task<IActionResult> Create([FromBody] CreateReservationDto dto)
{

```

```

summary.
    // Method: Create - Enforces FAT service 7-day booking validation rule and issues booking
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var currentNic = User.FindFirstValue(ClaimTypes.NameIdentifier);
    var currentRole = User.FindFirstValue(ClaimTypes.Role);

    // Prosumers can only create reservations for their own NIC
    if (currentRole == "Prosumer" && !string.IsNullOrEmpty(currentNic) && currentNic !=
dto.ProsumerNic)
    {
        return Forbid("Cannot create reservation on behalf of another prosumer.");
    }
    try
    {
        var reservation = await _reservationService.CreateReservationAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id = reservation.Id }, new
        {
            message = "Energy reservation confirmed successfully.",
            reservation,
            summary = new
            {
                reservation.ReservationNumber,
                reservation.StationName,
                reservation.ScheduledDateTime,
                reservation.EnergyAmountKwh,
                reservation.TradeType,
                reservation.Status,
                reservation.QrCodeToken
            }
        });
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

```

File: `backend/Controllers/ReservationsController.cs` – `Update`

```

public async Task<IActionResult> Update(string id, [FromBody] UpdateReservationDto dto)
{
    // Method: Update - Modifies booking schedule while validating 12-hour advance notice
    requirement.

```

```

        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }
        var currentNic = User.FindFirstValue(ClaimTypes.NameIdentifier) ?? string.Empty;
        var currentRole = User.FindFirstValue(ClaimTypes.Role) ?? string.Empty;
        try
        {
            var requestingNic = currentRole == "Prosumer" ? currentNic : string.Empty;
            var updated = await _reservationService.UpdateReservationAsync(id, requestingNic, dto);

            return Ok(new
            {
                message = "Reservation updated successfully.",
                reservation = updated,
                summary = new
                {
                    updated.ReservationNumber,
                    updated.StationName,
                    updated.ScheduledDateTime,
                    updated.EnergyAmountKwh,
                    updated.TradeType,
                    updated.Status,
                    updated.QrCodeToken
                }
            });
        }
        catch (ArgumentException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch (UnauthorizedAccessException ex)
        {
            return Forbid(ex.Message);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
    }
}

```

File: `backend/Controllers/ReservationsController.cs` – `Cancel`

```

public async Task<IActionResult> Cancel(string id, [FromBody] CancelReservationDto dto)
{
    // Method: Cancel - Enforces 12-hour cancellation notice rule, with Grid Operator assistance support.

    var currentNic = User.FindFirstValue(ClaimTypes.NameIdentifier) ?? string.Empty;
    var currentRole = User.FindFirstValue(ClaimTypes.Role) ?? string.Empty;

```

```

        try
        {
            var success = await _reservationService.CancelReservationAsync(id, currentNic,
currentRole, dto.Reason);
            if (!success)
            {
                return NotFound(new { message = "Reservation not found." });
            }
            return Ok(new
            {
                message = "Reservation cancelled successfully.",
                summary = new
                {
                    reservationId = id,
                    status = "Cancelled",
                    reason = dto.Reason,
                    cancelledAt = DateTime.UtcNow
                }
            });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
    }
}

```

File: `backend/Controllers/ReservationsController.cs` – `Approve`

```

public async Task<IActionResult> Approve(string id)
{
    // Method: Approve - Transitions reservation from Pending to Approved and issues cryptographic
    QR code pass.
    try
    {
        var approved = await _reservationService.ApproveReservationAsync(id);
        return Ok(new
        {
            message = "Reservation approved successfully. Cryptographic QR pass generated.",
            reservation = approved
        });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

```

File: ``mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/prosumer/CreateReservationActivity.java``
– 7-day booking flow

```
private void setupSubmitListener() {
    btnConfirmBooking.setOnClickListener(v -> {
        if (!isDateSelected || !isTimeSelected) {
            Toast.makeText(this, "Please select both date and time (within 7 days).",
                Toast.LENGTH_SHORT).show();
            return;
        }
        long diffMillis = selectedCalendar.getTimeInMillis() - System.currentTimeMillis();
        if (diffMillis < -600000) { // more than 10 mins in past
            com.ead.solarmicrogrid.util.SolvanceDialog.showWarning(
                this,
                "Invalid Appointment Time",
                "SCHEDULE POLICY",
                "Cannot schedule bookings in the past. Please select a future time slot within 7
                days.",
                "Adjust Time",
                null
            );
            return;
        }

        if (stationList.isEmpty() || spStations.getSelectedItemPosition() < 0) {
            Toast.makeText(this, "No solar hub selected.", Toast.LENGTH_SHORT).show();
            return;
        }
        String energyStr = etEnergyAmount.getText() != null ?
etEnergyAmount.getText().toString().trim() : "";
        double parsedEnergy;
        try {
            parsedEnergy = Double.parseDouble(energyStr);
        } catch (NumberFormatException e) {
            Toast.makeText(this, "Enter valid energy amount.", Toast.LENGTH_SHORT).show();
            return;
        }
        if (parsedEnergy <= 0) {
            Toast.makeText(this, "Energy amount must be greater than 0.", Toast.LENGTH_SHORT).show();
            return;
        }
        final double energy = parsedEnergy;

        SolarStation selectedStation = stationList.get(spStations.getSelectedItemPosition());
        String tradeType = rbDropOff.isChecked() ? "DropOff" : "Charging";

        SimpleDateFormat isoFormat = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss'Z'", Locale.US);
        isoFormat.setTimeZone(java.util.TimeZone.getTimeZone("UTC"));
        String scheduledIso = isoFormat.format(selectedCalendar.getTime());

        SimpleDateFormat localDisplayFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm (EEE)",
            Locale.getDefault());
        String localDisplayTime = localDisplayFormat.format(selectedCalendar.getTime());
    });
}
```

```

progressBar.setVisibility(View.VISIBLE);
btnConfirmBooking.setEnabled(false);

AuthDtos.CreateReservationRequest request = new AuthDtos.CreateReservationRequest(
    currentUser.getNic(),
    selectedStation.getId(),
    scheduledIso,
    energy,
    tradeType
);
ApiClient.getService(this).createReservation(request).enqueue(new Callback<ResponseBody>() {
    @Override
    public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
        progressBar.setVisibility(View.GONE);
        btnConfirmBooking.setEnabled(true);

        if (response.isSuccessful()) {
            java.util.List<com.ead.solarmicrogrid.util.SolvanceDialog.DetailItem> details =
new java.util.ArrayList<>();
            details.add(new com.ead.solarmicrogrid.util.SolvanceDialog.DetailItem("Station",
selectedStation.getName()));
            details.add(new com.ead.solarmicrogrid.util.SolvanceDialog.DetailItem("Schedule
Time", localDisplayTime));
            details.add(new com.ead.solarmicrogrid.util.SolvanceDialog.DetailItem("Energy
Quota", energy + " kWh"));
            details.add(new com.ead.solarmicrogrid.util.SolvanceDialog.DetailItem("Trading
Mode", tradeType));

            com.ead.solarmicrogrid.util.SolvanceDialog.showModal(
                CreateReservationActivity.this,
                com.ead.solarmicrogrid.util.SolvanceDialog.DialogType.SUCCESS,
                "Reservation Submitted",
                "AWAITING APPROVAL",
                "Your power trading reservation is pending operator approval. The secure
QR pass will appear after approval.",
                details,
                "After approval, your dynamic security QR will be available in the
reservation details.",
                "View Booking Details",
                null,
                false,
                () -> {
                    finish();
                    overridePendingTransition(R.anim.slide_in_left,
R.anim.slide_out_right);
                },
                null
            );
        } else {
            String errorMsg = "Booking rejected: Verify 7-day rule and account status.";
            try {
                if (response.errorBody() != null) {
                    String raw = response.errorBody().string();

```

```

        try {
            org.json.JSONObject obj = new org.json.JSONObject(raw);
            if (obj.has("message")) errorMsg = obj.getString("message");
            else errorMsg = raw;
        } catch (Exception ex) {
            if (!raw.isEmpty()) errorMsg = raw;
        }
    }
} catch (Exception ignored) {}
com.ead.solarmicrogrid.util.SolvanceDialog.showWarning(
    CreateReservationActivity.this,
    "Reservation Request Blocked",
    "SCHEDULE RULE",
    errorMsg,
    "Review & Retry",
    null
);
}
}
@Override
public void onFailure(Call<ResponseBody> call, Throwable t) {
    progressBar.setVisibility(View.GONE);
    btnConfirmBooking.setEnabled(true);
    Toast.makeText(CreateReservationActivity.this, "Network error: " + t.getMessage(),
Toast.LENGTH_LONG).show();
}
});
});
}
}

```

File: ``mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/prosumer/ReservationDetailActivity.java``
- 12-hour notice calculation

```

private double getHoursRemainingUntilBooking() {
    if (scheduledDateTime == null || scheduledDateTime.isEmpty()) return 999.0;
    try {
        java.util.Date parsed = parseDateSafely(scheduledDateTime);
        if (parsed != null) {
            long diffMillis = parsed.getTime() - System.currentTimeMillis();
            return diffMillis / (1000.0 * 60.0 * 60.0);
        }
    } catch (Exception ignored) {}
    return 999.0;
}

```

File: ``mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/prosumer/ReservationDetailActivity.java``
- cancellation request

```

private void executeCancel() {
    if (reservationId == null || reservationId.isEmpty()) {
        reservationId = resNumber;
    }
    if (reservationId == null || reservationId.isEmpty()) {
        Toast.makeText(this, "Reservation identifier not found.", Toast.LENGTH_SHORT).show();
    }
}

```

```

        return;
    }
    progressBar.setVisibility(View.VISIBLE);
    btnCancelReservation.setEnabled(false);

    AuthDtos.CancelRequest request = new AuthDtos.CancelRequest("Prosumer self-cancelled via mobile
app");
    ApiClient.getService(this).cancelReservation(reservationId, request).enqueue(new
Callback<ResponseBody>() {
        @Override
        public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
            progressBar.setVisibility(View.GONE);

            if (response.isSuccessful()) {
                btnCancelReservation.setEnabled(false);
                btnCancelReservation.setAlpha(0.4f);
                status = "Cancelled";
                tvStatusDetail.setText("Cancelled");
                applyStatusBadgeStyle(tvStatusDetail, "Cancelled");
                cardQr.setVisibility(View.GONE);

                com.ead.solarmicrogrid.util.SolvanceDialog.showSuccess(
                    ReservationDetailActivity.this,
                    "Cancellation Completed",
                    "BOOKING CANCELLED",
                    "Your reservation slot has been released back to the microgrid capacity
pool.\n\n12-hour cancellation notice policy was satisfied.",
                    null,
                    "Back to Reservations",
                    () -> {
                        finish();
                        overridePendingTransition(R.anim.slide_in_left, R.anim.slide_out_right);
                    }
                );
            } else {
                btnCancelReservation.setEnabled(true);
                String errorMsg = "Cancellations require at least 12 hours' notice before scheduled
appointment.";
                try {
                    if (response.errorBody() != null) {
                        String raw = response.errorBody().string();
                        try {
                            org.json.JSONObject obj = new org.json.JSONObject(raw);
                            if (obj.has("message")) {
                                errorMsg = obj.getString("message");
                            } else {
                                errorMsg = raw;
                            }
                        } catch (Exception ex) {
                            if (!raw.isEmpty()) errorMsg = raw;
                        }
                    }
                } catch (Exception ignored) {}
                com.ead.solarmicrogrid.util.SolvanceDialog.showWarning(

```

```

        ReservationDetailActivity.this,
        "Cancellation Blocked",
        "POLICY RESTRICTION",
        errorMsg + "\n\nModifications and cancellations require at least 12 hours'
advance notice per Microgrid policy. Please contact your Grid Operator for assistance if needed.",
        "Understood",
        null
    );
}
}
@Override
public void onFailure(Call<ResponseBody> call, Throwable t) {
    progressBar.setVisibility(View.GONE);
    btnCancelReservation.setEnabled(true);
    Toast.makeText(ReservationDetailActivity.this, "Network error: " + t.getMessage(),
Toast.LENGTH_SHORT).show();
}
});
}

```

File: ``web-client/src/pages/OperatorDashboard.jsx`` – reservation/operational data loading

```

const loadOperationalData = async () => {
    setIsSyncing(true);
    const currentStationId = selectedStationIdRef.current;
    try {
        const stationRes = await api.get('/stations');
        const stationList = stationRes.data || [];
        setStations(stationList);
        // Default the terminal to the first active hub on first load
        const activeStation = currentStationId
            ? stationList.find((s) => s.id === currentStationId)
            : stationList.find((s) => s.isActive) || stationList[0];
        if (!currentStationId && activeStation) {
            setSelectedStationId(activeStation.id);
        }
        // Keep slot counters in step with the server unless the operator has unsaved edits
        if (activeStation && !slotsDirtyRef.current) {
            setAvailableSlots(activeStation.availableBatterySlots);
            setTotalSlots(activeStation.totalBatterySlots);
        }
    } catch (err) {
        console.error('Failed to load solar stations', err);
    }

    try {
        const resRes = await api.get('/reservations');
        setReservations(resRes.data || []);
    } catch (err) {
        console.error('Failed to load reservations', err);
    } finally {
        // Only the first load shows the full-table spinner; background refreshes keep the table on screen
        setLoadingReservations(false);
    }
}

```

```

    if (currentStationId) {
      try {
        const telemetryRes = await api.get(`/stations/${currentStationId}/telemetry`);
        setTelemetry(telemetryRes.data);
      } catch (err) {
        console.error('Failed to load station telemetry', err);
      }
    }
  }
  setIsSyncing(false);
};

```

File: `web-client/src/pages/OperatorDashboard.jsx` – reservation approval and QR-pass issuance

```

const handleApproveReservation = async (res) => {
  if (approvingId) return;
  setApprovingId(res.id);
  try {
    const apiRes = await api.post(`/reservations/${res.id}/approve`);
    const updated = apiRes.data?.reservation || {};
    const fullPassData = { ...res, ...updated, status: 'Approved' };
    loadOperationalData();

    // The backend signs a QR token on approval; never show a made-up token that can't be verified
    if (!fullPassData.qrCodeToken) {
      showToast(`Reservation #${fullPassData.reservationNumber} approved. Refresh to load its QR pass.`,
        'warning');
      return;
    }
    setApprovedPassData(fullPassData);
    setGeneratedQrDataUrl('');
    setIsPassModalOpen(true);
    showToast(`Reservation #${fullPassData.reservationNumber} approved. QR pass issued.`, 'success');

    // Draw the QR image separately: a rendering failure must not look like a failed approval
    try {
      const qrDataUrl = await QRCode.toDataURL(fullPassData.qrCodeToken, {
        width: 320,
        margin: 2,
        color: {
          dark: '#0f172a',
          light: '#ffffff'
        }
      });
      setGeneratedQrDataUrl(qrDataUrl);
    } catch (qrErr) {
      console.error('Failed to render QR pass image', qrErr);
    }
  } catch (err) {
    showToast(err.response?.data?.message || 'Failed to approve reservation.', 'error');
  } finally {
    setApprovingId(null);
  }
};

```

7.4.H.N. Madubashini (IT23192300) - Operator Terminal, QR & Telemetry

Backend - QR verification and telemetry

File: ``backend/Controllers/ReservationsController.cs` - `VerifyQr``

```
public async Task<IActionResult> VerifyQr([FromBody] VerifyQrRequestDto dto)
{
    // Method: VerifyQr - Scans prosumer QR payload, checks server state, and finalizes energy
    transfer.
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }
    var operatorNic = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (string.IsNullOrEmpty(operatorNic))
    {
        return Unauthorized(new { message = "Authenticated operator identity is required." });
    }
    try
    {
        var completed = await _reservationService.VerifyAndFinalizeQrAsync(dto, operatorNic);
        return Ok(new
        {
            message = "Energy transfer verified and job completed successfully.",
            reservation = completed,
            summary = new
            {
                completed.ReservationNumber,
                completed.ProsumerNic,
                completed.StationName,
                completed.EnergyAmountKwh,
                completed.TradeType,
                completed.Status,
                completed.CompletedAt
            }
        });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (UnauthorizedAccessException ex)
    {
        return Unauthorized(new { message = ex.Message });
    }
}
```

File: ``backend/Services/ReservationService.cs`` – ``VerifyAndFinalizeQrAsync``

```
public async Task<EnergyReservation> VerifyAndFinalizeQrAsync(VerifyQrRequestDto dto, string operatorNic)
{
    // Method: VerifyAndFinalizeQrAsync - Operator mode endpoint that verifies scanned QR payload
    and finalizes energy transfer.
    var token = dto.QrCodeToken?.Trim() ?? string.Empty;
    if (string.IsNullOrEmpty(token))
    {
        throw new ArgumentException("QR Code token is required.");
    }
    if (!IsValidWellFormedQrToken(token))
    {
        throw new InvalidOperationException("Invalid QR token format/signature.");
    }
    var reservation = await _context.Reservations
        .Find(r => r.QrCodeToken == token)
        .FirstOrDefaultAsync();

    if (reservation == null)
    {
        throw new KeyNotFoundException("Invalid QR Code. No matching reservation record found.");
    }
    if (!IsValidQrSignature(reservation, token))
    {
        throw new InvalidOperationException("Invalid QR token format/signature.");
    }
    if (!string.IsNullOrEmpty(dto.StationId) &&
        !string.Equals(reservation.StationId, dto.StationId.Trim(),
            StringComparison.OrdinalIgnoreCase))
    {
        throw new InvalidOperationException("QR token does not belong to the selected station.");
    }
    if (reservation.Status != "Approved")
    {
        throw new InvalidOperationException($"Transaction cannot be finalized. Current status is
        '{reservation.Status}' (Expected: 'Approved').");
    }
    var now = DateTime.UtcNow;
    var opNic = !string.IsNullOrEmpty(operatorNic) ? operatorNic.Trim() : "OPERATOR";
    var update = Builders<EnergyReservation>.Update
        .Set(r => r.Status, "Completed")
        .Set(r => r.CompletedAt, now)
        .Set(r => r.CompletedByOperatorNic, opNic)
        .Set(r => r.UpdatedAt, now);

    await _context.Reservations.UpdateOneAsync(r => r.Id == reservation.Id, update);
    reservation.Status = "Completed";
    reservation.CompletedAt = now;
    reservation.CompletedByOperatorNic = opNic;

    if (reservation.SlotReserved && !string.IsNullOrEmpty(reservation.SlotId))
    {
        await ReleaseSlotAsync(reservation);
    }
}
```

```

        // Update station available battery slot if charging/dropoff completed
        if (!string.IsNullOrEmpty(reservation.StationId))
        {
            var station = await _context.Stations.Find(s => s.Id ==
reservation.StationId).FirstOrDefaultAsync();
            if (station != null)
            {
                if (reservation.TradeType == "DropOff" && station.AvailableBatterySlots > 0)
                {
                    var slotUpdate = Builders<SolarStation>.Update.Inc(s => s.AvailableBatterySlots, -
1);

                    await _context.Stations.UpdateOneAsync(s => s.Id == station.Id, slotUpdate);
                }
                else if (reservation.TradeType == "Charging" || reservation.TradeType == "PickUp")
                {
                    var availableSlots = Math.Min(station.TotalBatterySlots,
station.AvailableBatterySlots + 1);
                    await _context.Stations.UpdateOneAsync(s => s.Id == station.Id,
Builders<SolarStation>.Update.Set(s => s.AvailableBatterySlots, availableSlots));
                }
            }
        }

        return reservation;
    }

```

File: `backend/Services/ReservationService.cs` - `GenerateSecureQrToken`

```

private string GenerateSecureQrToken(EnergyReservation reservation)
{
    // Method: GenerateSecureQrToken - Creates tamper-resistant signed string for mobile QR
    rendering and operator scanning.
    var raw =
    $"{reservation.ReservationNumber}|{reservation.ProsumerNic}|{reservation.StationId}|{reservation.Scheduled
    DateTime:yyyyMMddHHmm}|{reservation.EnergyAmountKwh}";
    using var hmac = new HMACSHA256(
    Encoding.UTF8.GetBytes(_qrSigningSecret));
    var hashBytes = hmac.ComputeHash(Encoding.UTF8.GetBytes(raw));
    var sig = Convert.ToHexString(hashBytes)[..12];
    return $"SOLAR-TX:{reservation.ReservationNumber}:{sig}";
}

```

File: `backend/Services/ReservationService.cs` - `ValidateQrSignature`

```

private bool ValidateQrSignature(EnergyReservation reservation, string token)
{
    if (!IsValidWellFormedQrToken(token))
    {
        return false;
    }
    var expectedToken = GenerateSecureQrToken(reservation);
    var expectedBytes = Encoding.UTF8.GetBytes(expectedToken);
    var actualBytes = Encoding.UTF8.GetBytes(token);
}

```

```

        return expectedBytes.Length == actualBytes.Length &&
            CryptographicOperations.FixedTimeEquals(expectedBytes, actualBytes);
    }

```

File: `backend/Controllers/StationsController.cs` - `GetTelemetry`

```

public async Task<IActionResult> GetTelemetry(string id)
{
    var telemetry = await _stationService.GetTelemetryAsync(id);
    return telemetry == null
        ? NotFound(new { message = "Station not found." })
        : Ok(telemetry);
}

```

File: `backend/Services/StationService.cs` - `GetTelemetryAsync`

```

public async Task<StationTelemetryDto?> GetTelemetryAsync(string id)
{
    var station = await GetStationByIdAsync(id);
    if (station == null)
    {
        return null;
    }

    var pending = await _context.Reservations.CountDocumentsAsync(r => r.StationId == id &&
        r.Status == "Pending");
    var approved = await _context.Reservations.CountDocumentsAsync(r => r.StationId == id &&
        r.Status == "Approved");
    var completed = await _context.Reservations.CountDocumentsAsync(r => r.StationId == id &&
        r.Status == "Completed");
    var occupied = Math.Max(0, station.TotalBatterySlots - station.AvailableBatterySlots);

    return new StationTelemetryDto
    {
        StationId = station.Id ?? id,
        StationCode = station.StationCode,
        StationName = station.Name,
        IsActive = station.IsActive,
        TotalBatterySlots = station.TotalBatterySlots,
        AvailableBatterySlots = station.AvailableBatterySlots,
        OccupiedBatterySlots = occupied,
        BatteryOccupancyPercent = station.TotalBatterySlots == 0 ? 0 : occupied * 100d /
            station.TotalBatterySlots,
        PendingReservations = (int)pending,
        ApprovedReservations = (int)approved,
        CompletedReservations = (int)completed,
        CapturedAt = DateTime.UtcNow
    };
}

```

Android client - Operator scanner

File: `mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/operator/OperatorScannerActivity.java` -
`setupBarcodeScanner`

```

private void setupBarcodeScanner(Bundle savedInstanceState) {
    barcodeScannerView.initializeFromIntent(getIntent());
    barcodeScannerView.setStatusText("");
    barcodeScannerView.getBarcodeView().setDecoderFactory(
        new DefaultDecoderFactory(Collections.singletonList(BarcodeFormat.QR_CODE))
    );
    barcodeScannerView.decodeContinuous(new BarcodeCallback() {
        @Override
        public void barcodeResult(BarcodeResult result) {
            if (result.getText() != null && !isProcessingScan && activeTab == TAB_SCANNER) {
                isProcessingScan = true;
                runOnUiThread(() -> verifyQrCodeOnServer(result.getText()));
            }
        }
        @Override
        public void possibleResultPoints(List<ResultPoint> resultPoints) {}
    });
}

```

File: ``mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/operator/OperatorScannerActivity.java`` – ``verifyQrCodeOnServer``

```

private void verifyQrCodeOnServer(String qrToken) {
    progressBarOperator.setVisibility(View.VISIBLE);
    cardVerifiedResult.setVisibility(View.GONE);

    String stationId = selectedStation != null ? selectedStation.getId() : null;
    AuthDtos.VerifyQrRequest request = new AuthDtos.VerifyQrRequest(qrToken, stationId);

    ApiClient.getService(this).verifyQr(request).enqueue(new Callback<ResponseBody>() {
        @Override
        public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
            progressBarOperator.setVisibility(View.GONE);

            if (response.isSuccessful()) {
                cardVerifiedResult.setVisibility(View.VISIBLE);
                tvResultDetails.setText("Verified Token: " + qrToken + "\nStatus: Completed & Grid
Finalized");
                Toast.makeText(OperatorScannerActivity.this, "Energy Transfer Completed & Verified!",
Toast.LENGTH_SHORT).show();
                fetchReservations();
            } else {
                Toast.makeText(OperatorScannerActivity.this, "Invalid or already completed QR code.",
Toast.LENGTH_LONG).show();
            }
            barcodeScannerView.postDelayed(() -> isProcessingScan = false, 3000);
        }
        @Override
        public void onFailure(Call<ResponseBody> call, Throwable t) {
            progressBarOperator.setVisibility(View.GONE);
            Toast.makeText(OperatorScannerActivity.this, "API Connection error: " + t.getMessage(),
Toast.LENGTH_LONG).show();
            barcodeScannerView.postDelayed(() -> isProcessingScan = false, 3000);
        }
    });
}

```

```

    }
    });
}

```

File: ``mobile-client/app/src/main/java/com/ead/solarmicrogrid/ui/operator/OperatorScannerActivity.java`` – ``syncBatterySlots``

```

private void syncBatterySlots() {
    if (selectedStation == null) {
        Toast.makeText(this, "No station selected.", Toast.LENGTH_SHORT).show();
        return;
    }
    progressBarOperator.setVisibility(View.VISIBLE);
    btnSyncBatteryStorage.setEnabled(false);

    AuthDtos.UpdateBatterySlotsRequest req = new
AuthDtos.UpdateBatterySlotsRequest(currentAvailableSlots);

    ApiClient.getService(this).updateBatterySlots(selectedStation.getId(), req).enqueue(new
Callback<ResponseBody>() {
        @Override
        public void onResponse(Call<ResponseBody> call, Response<ResponseBody> response) {
            progressBarOperator.setVisibility(View.GONE);
            btnSyncBatteryStorage.setEnabled(true);

            if (response.isSuccessful()) {
                selectedStation.setAvailableBatterySlots(currentAvailableSlots);
                Toast.makeText(OperatorScannerActivity.this, "Station storage synchronized with
Central Grid!", Toast.LENGTH_SHORT).show();
                fetchStations();
            } else {
                String errorMsg = "Failed to update storage slots (" + response.code() + ")";
                try {
                    if (response.errorBody() != null) {
                        errorMsg += ": " + response.errorBody().string();
                    }
                } catch (Exception ignored) {}
                Toast.makeText(OperatorScannerActivity.this, errorMsg, Toast.LENGTH_LONG).show();
            }
        }
        @Override
        public void onFailure(Call<ResponseBody> call, Throwable t) {
            progressBarOperator.setVisibility(View.GONE);
            btnSyncBatteryStorage.setEnabled(true);
            Toast.makeText(OperatorScannerActivity.this, "Network error: " + t.getMessage(),
Toast.LENGTH_LONG).show();
        }
    });
}

```

Web client - Telemetry and QR verification

File: ``web-client/src/pages/OperatorDashboard.jsx`` – telemetry-aware station data and KPI source

```

const loadOperationalData = async () => {
  setIsSyncing(true);
  const currentStationId = selectedStationIdRef.current;
  try {
    const stationRes = await api.get('/stations');
    const stationList = stationRes.data || [];
    setStations(stationList);
    // Default the terminal to the first active hub on first load
    const activeStation = currentStationId
      ? stationList.find((s) => s.id === currentStationId)
      : stationList.find((s) => s.isActive) || stationList[0];
    if (!currentStationId && activeStation) {
      setSelectedStationId(activeStation.id);
    }
    // Keep slot counters in step with the server unless the operator has unsaved edits
    if (activeStation && !slotsDirtyRef.current) {
      setAvailableSlots(activeStation.availableBatterySlots);
      setTotalSlots(activeStation.totalBatterySlots);
    }
  } catch (err) {
    console.error('Failed to load solar stations', err);
  }
  try {
    const resRes = await api.get('/reservations');
    setReservations(resRes.data || []);
  } catch (err) {
    console.error('Failed to load reservations', err);
  }
  finally {
    // Only the first load shows the full-table spinner; background refreshes keep the table on screen
    setLoadingReservations(false);
  }
  if (currentStationId) {
    try {
      const telemetryRes = await api.get(`/stations/${currentStationId}/telemetry`);
      setTelemetry(telemetryRes.data);
    } catch (err) {
      console.error('Failed to load station telemetry', err);
    }
  }
  setIsSyncing(false);
};

```

File: ``web-client/src/pages/OperatorDashboard.jsx`` – direct QR verification

```

const handleDirectVerifyQr = async (tokenToVerify) => {
  const rawToken = (tokenToVerify || qrToken || '').trim();
  if (!rawToken) {
    setQrError('Enter or scan a valid QR transaction token.');
```

return;

```

  }
  setVerifying(true);
  setQrError('');
  setVerifiedResult(null);
  try {

```

```

const payload = {
  qrCodeToken: rawToken,
  // Ref: this can run from the camera loop, whose closure may predate a hub change
  stationId: selectedStationIdRef.current || undefined
};

const res = await api.post('/reservations/verify-qr', payload);
const resultData = res.data.summary || res.data.reservation || res.data;

setVerifiedResult(resultData);
setQrToken('');
setInspectingPass(null);
showToast(`Transfer completed. Pass verified by ${user?.fullName || 'Operator'}.`, 'success');
loadOperationalData();
} catch (err) {
  const errMsg = err.response?.data?.message || 'Verification failed. Invalid token, station mismatch,
or already completed.';
  setQrError(errMsg);
  showToast(errMsg, 'error');
} finally {
  setVerifying(false);
}
};

```

8. Individual Contribution

IT Number	Assigned Role & Contributions	AI Tools Disclosed & Usage Reflection
IT23452916 - M.L.BOOSO	Primary Domain: User Identity & Prosumer Lifecycle Management - Business Logic: Implemented the prosumer registration and account lifecycle, including NIC uniqueness validation and the Pending → Active approval workflow. - Backend & Web API: Developed the UserService and UsersController functionality for user registration, authentication-related operations, profile/status management, and prosumer administration. - UI Implementation: Developed the Login.jsx and ProsumerManagement.jsx views for the Web Portal, together with the Android LoginActivity and RegisterActivity screens. - Security: Implemented password hashing and role-based/JWT authentication configuration for protected application functionality.	AI Tools: Used AI assistance for code-level research, debugging, documentation, and understanding implementation approaches. Usage Reflection: AI was used as a supporting tool rather than as a replacement for development. Generated suggestions were reviewed, adapted to the project's requirements, and tested before being incorporated.
IT23151260 - G.L.S.CHANLAKA	Primary Domain: Solar Nodes, Stations & Geolocation Management - Business Logic: Implemented the station deactivation blocker that prevents deactivation when active energy reservations are associated with the station. - Backend & Web API: Developed the StationService and SlotService functionality and corresponding controller operations for station CRUD, station lifecycle management, and battery-slot management. - UI Implementation: Developed the NodeManagement.jsx view for Web Portal station/slot management and implemented station location visualization and selection through StationsMapActivity. - Mobile Integration: Integrated station geolocation and map-based station selection into the mobile reservation workflow.	AI Tools: Used AI assistance for implementation guidance, debugging, code explanation, and documentation support. Usage Reflection: AI-generated suggestions were treated as references and were manually reviewed and tested against the project's station, slot, and reservation requirements.
IT23156760 - L.T.JAYAWARDHANA	Primary Domain: Energy Reservations & Business Rules	AI Tools: Used AI assistance for code

	• Business Logic: Implemented the reservation lifecycle and business rules, including the 7-day advance booking rule and 12-hour modification/cancellation restriction. • Backend & Web API: Developed the reservation functionality in ReservationService and ReservationsController for creating, updating, cancelling, and approving energy reservations. • UI Implementation: Developed the reservation-related sections of the Web Portal operator dashboard and the Android reservation flow, including station/slot selection and reservation creation. • Database: Implemented the application logic responsible for managing energy reservation records through the backend persistence layer.	understanding, debugging, implementation guidance, and documentation preparation. Usage Reflection: AI was used to accelerate development and investigate implementation issues. The resulting code and suggestions were reviewed and validated against the defined reservation rules before use.
IT23192300 - H.N.MADUBASHINI	Primary Domain: Operator Terminal, QR Verification & Telemetry • Business Logic: Implemented secure QR-token generation, signature validation, and server-side QR verification/finalization for on-site energy transaction processing. • Backend & Web API: Developed the QR verification functionality in ReservationService and the reservation verification endpoint, together with station telemetry retrieval and real-time battery-slot synchronization functionality. • UI Implementation: Developed the telemetry and QR-verification portions of OperatorDashboard.jsx and implemented camera-based QR scanning in OperatorScannerActivity. • Mobile Integration: Implemented QR scanning, server verification, battery-slot synchronization, and operator cancellation-override functionality.	AI Tools: Used AI assistance for security-focused code research, debugging, implementation guidance, and documentation support. Usage Reflection: AI was used as a development aid for understanding cryptographic verification, API integration, and debugging.

9. Challenges

9.1.Challenges Faced

- Client–Server Integration - Integrating the web application and native Android application with the centralized C# Web API was challenging because both clients had to communicate consistently with the same backend services and handle API responses correctly.
- MongoDB Database Integration - Designing and connecting the required MongoDB collections with the Web API required careful handling of data structures and relationships between user details, solar stations, energy booking slots, and energy reservations.
- Reservation Business Rules - Implementing the reservation rules was challenging, particularly the requirement that reservations must be scheduled within 7 days and that updates and cancellations require at least 12 hours' notice. These rules had to be handled consistently through the service layer.
- Native Android and SQLite Implementation - Developing the mobile application using pure native Android while maintaining local data using SQLite required additional effort compared with using cross-platform frameworks.
- QR Code Verification - Implementing secure QR-based transaction verification was challenging because the QR code had to represent the approved transaction and be verified before the energy transfer could be finalized.
- Google Maps Integration - Integrating Google Maps to display and locate nearby microgrid nodes required handling location data and displaying the relevant station information correctly within the mobile application.

9.2.Solutions

The challenges were addressed by separating responsibilities between the clients and the centralized Web API. Business rules were implemented within the API service layer, while MongoDB was used for centralized data management and SQLite was used for required local Android persistence. The team also divided the system into separate functional areas so that each member could focus on specific components while maintaining integration between the modules.

10. Conclusion

The Smart Solar Microgrid Trading System provides an integrated client-server solution for managing solar energy trading activities within a microgrid environment. The system combines a responsive web application for back-office and grid operator activities with a native Android mobile application for solar prosumers and grid operators. Both client applications communicate with a centralized C# Web API, which manages the application's business logic and interaction with the database.

The system covers the main requirements of the assignment, including user and prosumer management, microgrid node management, energy slot reservations, reservation updates and cancellations, booking history, map-based microgrid node identification, and QR-based transaction verification. The use of a centralized API ensures that the core business rules are handled consistently across the web and mobile applications, while SQLite provides local persistence within the Android application where required.

Overall, the application demonstrates the use of client-server architecture to support multiple client platforms while maintaining centralized business logic and data management. The developed solution provides the required foundation for managing solar energy reservations and transactions between solar prosumers and microgrid operators.

11. References

- [1] Microsoft, “ASP.NET Core Web API,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/web-api/>. [Accessed: Oct. 5, 2026].
- [2] Microsoft, “ASP.NET Core authentication,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/>. [Accessed: Oct. 5, 2026].
- [3] MongoDB, “C#/.NET Driver Documentation,” MongoDB Documentation. [Online]. Available: <https://www.mongodb.com/docs/drivers/csharp/>. [Accessed: Oct. 5, 2026].
- [4] Microsoft, “System.Security.Cryptography Namespace,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography>. [Accessed: Oct. 5, 2026].
- [5] Android Developers, “Save data in a local database using SQLite,” Android Developers. [Online]. Available: <https://developer.android.com/training/data-storage/sqlite>. [Accessed: Oct. 5, 2026].
- [6] Square, “Retrofit,” GitHub. [Online]. Available: <https://square.github.io/retrofit/>. [Accessed: Oct. 5, 2026].
- [7] Google Developers, “Maps SDK for Android,” Google for Developers. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk>. [Accessed: Oct. 5, 2026].
- [8] Google Developers, “Google Maps Embed API,” Google for Developers. [Online]. Available: <https://developers.google.com/maps/documentation/embed/>. [Accessed: Oct. 5, 2026].
- [9] React, “React Documentation,” React. [Online]. Available: <https://react.dev/>. [Accessed: Oct. 5, 2026].
- [10] Tailwind CSS, “Tailwind CSS Documentation,” Tailwind CSS. [Online]. Available: <https://tailwindcss.com/>. [Accessed: Oct. 5, 2026].
- [11] Axios, “Axios Documentation,” Axios. [Online]. Available: <https://axios-http.com/docs/intro>. [Accessed: Oct. 5, 2026].
- [12] ZXing, “ZXing: Multi-format 1D/2D barcode image processing library,” GitHub. [Online]. Available: <https://github.com/zxing/zxing>. [Accessed: Oct. 5, 2026].

12. Appendix

